

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\agent-a9c0266d36acee0d2.jsonl`  
- SHA-256: `e310c4828b48f263ea73845ea736f91b3cfc73f395b2dc12d60d4d05f1fb62b8`  
- Source modified: `2026-06-07T06:57:07+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `subagents`  
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`
```

Transcript

1. user (2026-06-07T06:51:36.677Z)

Design an implementation plan for a "rich per-video observability report" feature spanning TWO existing Python repos plus a NEW shared private library repo. I've already done deep exploration; your job is to harden the design, detail file-level integration hooks, surface risks, and propose a phasing that won't become a "house of cards." Do NOT write code ? produce a concrete design/plan with trade-offs.

Goal (from the user)

After each tool processes/ingests a video, it must ALWAYS emit a rich "side output" report capturing every stage the video went through + details (fps, resolution, frames extracted, candidate windows, rallies/segments, AI timing, validation results, etc.), designed to (a) catch inconsistencies, (b) evaluate output differences across runs/tools, and (c) let someone who has only read the README + FAQ self-debug a bad run. Also a SECOND "customer-friendly" report that strips system internals and shows only video metadata + meta-level highlights. Both tools produce both report types. Reports are ALWAYS written, by convention next to the video file by default, with a configurable override location. The schema should be unified via a NEW separate private GitHub repo providing a reusable "report config language" consumed by both tools today and others later.

Constraints

- Permissive licensing ONLY (MIT/BSD/Apache-2.0/CC0) ? this is a hard commercial constraint (their ADR-005). NO copyleft, NO ELv2.
- Research already done: Soda Core/SodaCL is ELv2 (disqualified). Great Expectations (Apache-2.0) + Evidently (Apache-2.0) are license-safe but heavy/tabular-ML-shaped ? wrong fit. ydata-profiling (MIT) + Frictionless (MIT) also dataframe/data-package shaped. Conclusion: build a SMALL in-house library on permissive primitives the repos ALREADY use: pydantic (MIT), Jinja2 (BSD, already an indexer dep), PyYAML (MIT, already a collector dep), jsonschema (MIT); optionally borrow thin MIT renderers (yaml-to-markdown, json-schema-for-humans). Validate or challenge this conclusion.
- User strongly prefers FREQUENT SMALL PRs and a robust, well-tested rollout. Both repos run pytest offline with mocked external deps.

Repo A ? badminton-highlight-indexer (the "highlights generation tool")

Pipeline (runtime): backend/main.py orchestrates. In both the CLI path (main.py ~lines 312-352) and the FastAPI path (process_video @POST /api/process, ~lines 88-134):

```
video_id = get_file_md5(path) -> registry.run_all(path, config)  
[backend/validators/base.py::ValidationRegistry.run_all] -> db.add_video(...) -> seg =  
segmenter_registry.get(name)(db,config) -> segments = seg.process_video(video_id, path,  
player_pool, max_frames) -> (compile step is separate, VideoCompiler.compile_highlights in  
backend/stitcher/compiler.py).
```

CRITICAL plumbing facts I found:

- registry.run_all() builds a per-run `context` dict containing ffprobe_meta (fps/resolution/etc., produced by FormatValidator) but DISCARDS it ? it returns only List[ValidationResult] (validator_name, passed, message, details:dict). To get probe metadata into a report you must either (a) modify run_all to also surface the context, or (b) re-probe.

- `segmenter.process_video()` returns ONLY the final List[play_segment dicts]. The rich per-phase internals are NOT returned: candidate windows are persisted to DB (`db.add_candidate_segment`, queryable via `db.query_candidate_segments(detector=)`); AI provider timing {upload_time, process_time, gen_time} is returned by `provider.analyze_video` but only used internally; WASB substrate writes output/wasb/.../manifest.json {frames_total, windows_total, windows_done} + a trajectory CSV (visible-shuttle %); compiler returns skipped tuples (idx, start, end, reason). Much detail is only print()-ed.
- Known footguns to encode as report "flags"/warnings (these ARE the inconsistencies to catch): three different default segmenters across `config.json(gemini)/setup.py(yolo_hybrid)/main.py(motion)` so the report must show which ACTUALLY ran; hybrids+gemini return [] (not error) if {PROVIDER}_API_KEY missing OR WSL healthcheck fails (silent no-op) -> flag "0 segments + no API key"; fps/frame_width fallback (30.0/1280.0) silently mis-scales -> flag probed-vs-fallback; motion segmenter FABRICATES metadata/events (random) -> flag as synthetic/not-ground-truth; `add_video` is INSERT OR REPLACE + FK CASCADE -> re-ingest deletes prior segments (cache hit vs fresh).

Data contracts are documented in `docs/CODE_MAP.md` (read it). Tests live in `tests/test_*.py` (pytest, mocked). `README.md` has a strong FAQ (Q1-Q9) the report should cross-reference for debuggability.

Repo B ? sports-data-collector (the "data collector tool")

CLI: `src/cli/curate.py` (SportsDataCLI) subcommands: `status`, `curate`, `import-local`, `convert-cvat`, `validate-delivery`, `fetch`, `export`. Store: SQLite `data/sports_metadata.db` (videos table + per-sport rally/point tables; schema in `src/core/db.py`). Canonical shapes in `src/core/schemas/*.EXISTING` report pattern to REUSE (do not duplicate): `src/parsers/cvat/rally_cvat.py` has `build_summary()` (JSON), `render_validation_md()` (MD), `write_review_csv()`, `detect_issues()` (severity/blocker logic incl possible_missed_rally gap check), `render_issue_report()`; `validate-delivery` writes `<id>_report.json.md` + `<id>_review.csv` (`curate.py` ~562-617). `export` writes `<sport>/<video_id>.csv` + `manifest.json` (`curate.py` ~428-524). Per-video "stages" for the collector = acquisition (`fetch`: url, download attempts, probed res/fps via `src/core/downloader.py`), parse (source format JSON/CSV/CVAT, n records), time-recompute (fps used, frame range; frame/fps), validation (existing `detect_issues`), registration (DB insert: `video_id/status/license/is_commercial`), export. Time is decimal seconds internally; mm:ss for humans (`_mmss`). Tests in `tests/test_*.py`.

What I need from you

1. ****Validate/refine the library-vs-framework decision.**** Confirm building a small in-house lib on `pydantic+Jinja2+PyYAML+jonschema` is right (or argue for an alternative), given the licensing + per-video-pipeline-stage shape + house-of-cards risk.
2. ****Design the shared library**** ("report config language"): propose a repo name; the typed report ENVELOPE (identity, stages[], metrics, flags/warnings, verdict, audience views technical|customer); the declarative "config language" (YAML/JSON spec describing sections, fields, audience, and DECLARATIVE RULES that turn footguns into flags ? e.g. "when `ai.api_key_present==false` and `ai.segments==0` -> error 'silent provider no-op, see FAQ Q7'"); the renderers (JSON source-of-truth + Markdown technical + Markdown/HTML customer); a RunRecorder helper for incremental in-flight capture AND a post-hoc DB/artifact harvest fallback; how it's pip-installed across two private repos (git+ssh). Keep it MINIMAL and well-tested.
3. ****Indexer integration**** ? exact hooks: how to surface `ffprobe_meta` from `run_all` (recommend a backward-compatible approach), where to instantiate the RunRecorder in BOTH the CLI and API paths, how to harvest candidates/play_segments/AI-timing/WASB-manifest/compiler-skips, where to write the files (convention: next to video; config override), and which footgun flags to encode.
4. ****Collector integration**** ? how to wrap the existing `build_summary/detect_issues` outputs into the shared envelope WITHOUT duplicating logic, across `import-local/convert-cvat/fetch/export`, and the customer "delivery summary".
5. ****README/FAQ-reader debuggability**** ? propose a concrete iteration/eval method (e.g. a "fresh reader" subagent given ONLY README+FAQ+report that must diagnose a deliberately-broken run; iterate wording/flags until diagnosis is correct; each flag cites a FAQ section).
6. ****Phasing into small PRs**** across the 3 repos with tests at each step, ordered to deliver value early and de-risk. Call out the riskiest parts and how to contain them.
7. ****Risks/edge cases****: remote videos (no local file) for the collector; report write failures must NEVER break the main run; schema versioning; idempotency/overwrite vs append; large reports.

Be concrete and cite file paths. Return a structured plan I can adapt into a final plan document.

2. assistant (2026-06-07T06:51:39.267Z)

I'll start by exploring both repos to validate the design facts and harden the plan. Let me read the key files in parallel.

3. user (2026-06-07T06:51:40.656Z)

```
1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack.
4
5      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change.
6
7      **LEGEND**
8      - `@path` = file (always repo-relative)
9      - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
10     - `->` = dataflow / call / "produces"
11     - `$` = data contract (canonical shape)
12     - `?` = gotcha / footgun
13     - `?` = registry / plugin extension point
14     - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
15
16     ---
17
18     ## 1. PIPELINE
19
20     ### 1A. Runtime: video file -> highlight reel
21     ```
22     config.json (json.load; absent -> {})
23     -> video_id = MD5(whole file) @backend/main.py::get_file_md5 (? chunk size
varies per endpoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE
file so values agree)
24     -> validate
@backend/validators/base.py::registry.run_all(video_path, config)
25     FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/validators/__init__.py)
26     -> db.add_video(video_id, filepath, status, [r.dict()]) @backend/database.py
27     -> seg = segmenter_registry.get(name)(db, config) ?
@backend/indexer/base.py::segmenter_registry
28     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
29     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
30     -> select segment ids -> [(start,end)] (+ stitching padding)
31     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/stitcher/compiler.py
32     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
33     ```
34     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
35     ```
36     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
37     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
38     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
39     -> P4 db.add_play_segment + db.link_candidate_to_segment
40     ```
41     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
42     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
```

```

truth).
43
44     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
45     ```
46     WasbHybridSegmenter.process_video @backend/indexer/wasb_hybrid.py
47     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/detectors/wasb_runner.py
48     -> WasbRunner.run_predict(video, out_dir)
49     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
50     -> @backend/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector
pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
51     -> TrackNetRunner.parse_trajectory_csv(csv) @backend/detectors/tracknet_runner.py
(shared, re-exported by WasbRunner)
52     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
53     ```
54     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
55
56     ### 1C. Calibration / eval flow (NO pipeline run unless told)
57     ```
58     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
59     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
60     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
61     -> harness.evaluate -> EvalReport -> report_to_dict
62     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
63     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
64     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
65     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
66     ```
67     Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
68
69     ---
70
71     ## 2. SUBSYSTEMS
72
73     ### 2.0 Orchestration & config
74     - `@backend/main.py` ? FastAPI app + argparse CLI (modes via FLAGS, not subcommands:
`--server`/`--video-path` (process)/`--debug-clip`/`--trim-start`+`--trim-end`/`--setup`).
75     - `::app` FastAPI; `::db_instance`, `::global_config` module globals set ONLY in
`main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
76     - `::process_video` `@POST /api/process` -> validate -> add_video -> segment.
`::compile_highlights` `@POST /api/compile`. `::run_cli_diagnose_clip` lazy `import cv2`,
samples ±3s vs `indexing.motion_threshold`.
77     - ? uvicorn host hardcoded `127.0.0.1`; CLI default_segmenter fallback `motion`.
78     - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), `default_segmenter` fallback `yolo_hybrid`, skips validation.
79     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
80     - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label
`collector_video_id`).
81     - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run).

```

```

82     - `@config.json` ? single config. Sections: `validation` (+ `.relevance`), `indexing` (+
`.tracknet`; ? NO `.wasb` block in checked-in config ? WASB runs use `WasbConfig` defaults),
`output`, `stitching`, root `sport/ai_provider/ai_model`. ? **three default segmenters
diverge**: config ships `gemini`, setup.py writes `yolo_hybrid`, main.py fallback `motion`.
83     - `@setup.py` ? write default config (skips if present), ffmpeg/torch/CUDA diagnostics,
GPU-aware pip (`nvidia-smi` -> cu124 else cpu). ? generated config diverges from checked-in (no
`relevance`/`tracknet` blocks, blur 100.0 vs 20.0).
84
85     ### 2.1 Segmenters (indexer) ?
86     - `@backend/indexer/base.py`
87     - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id, start_time, end_time, duration, metadata}]`.
88     - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/indexer/__init__.py` (registration is an IMPORT side-
effect). ? `register()` builds `cls(None, {})` just to read `name` -> name/description MUST NOT
touch `self.db`/`self.config`.
89     - `@backend/indexer/__init__.py` ? imports all 5
(`motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid`) = the de-facto registration manifest.
90     - `@backend/indexer/wasb_hybrid.py::WasbHybridSegmenter` (name `wasb_hybrid`) ? WSL
WASB substrate. Config `indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; hardcoded
relative out dir `output/wasb`; needs `{PROVIDER}_API_KEY` (missing -> []). Registered at module
bottom.
91     - `@backend/indexer/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid (swaps runner; `detection_params` adds
`queue_length`). ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
92     - `@backend/indexer/yolo_hybrid.py::HybridYoloSegmenter` (name `yolo_hybrid`, no YOLO
remains ? torchvision detector + supervision ByteTrack). `::PERSON_CLASS_ID=1` ? (ultralytics
used 0; wrong value -> zero detections). `::lazy_load_model`,
`::extract_action_windows_from_chunk`,
`::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity computed at
`effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; trackers
per-chunk (windows merged by TIME not ID); `import torch` at module top.
93     - `@backend/indexer/gemini.py::GeminiPlaySegmenter` (name `gemini`) ? pure-AI, no P2.
? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded (ignores
`ai_max_workers`); `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates
analysis; `parse_time` accepts colon timestamps (warns).
94     - `@backend/indexer/motion.py::MotionPlaySegmenter` (name `motion`) ? pixel absdiff
baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
95
96     ### 2.2 Detectors & windowing (WSL-quarantined)
97     - `@backend/detectors/__init__.py` ? docstring only (refs `docs/DECISIONS.md` ADR-001).
? does NOT re-export runners; import submodules directly.
98     - `@backend/detectors/tracknet_runner.py`
99     - `::TrackNetConfig` (+`.from_indexing_cfg`), `::TrackNetRunner` ? runs TrackNet
`predict.py` in WSL.
100    - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`), `::TrajectoryPoint`(frame, visible, x, y).
101    - `::parse_trajectory_csv` @staticmethod** + `::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `` -> `run_predict` returns None; predict.py only `.mp4/.avi`; `visible==False`
resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent;
`track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever).
102    - `@backend/detectors/wasb_runner.py::WasbRunner` (+`::WasbConfig`) ? stages
`wasb_infer.py` + video into WSL, runs in `wasb` env.
`::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)`. ?
`run_predict` does NOT pass cache flags (uses wasb_infer defaults flush=1000/batch=8/workers=4);
`wasb_infer.py` MUST be re-staged each run (editing Windows copy is inert until staged);
`wsl_bash` duplicated (drift risk).
103    - `@backend/detectors/wasb_infer.py` (WSL) ? `::run` core. Reboot-resumable detector
cache: `::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`,

```

```

`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible`,
`::atomic_write_text/_atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be np.array). ?
GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `runner.gpus=[0]` hardcoded
single-GPU; `extract_frames` cv2-PNG SLOW (prefer host ffmpeg).
104
105     ### 2.3 Eval & calibration (all pure; I/O lives in CLI drivers)
106     - `@backend/eval/metrics.py` ? scoring primitive. `::interval_iou`, `::match_intervals`
(greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`,
`::Match`, `::MatchResult`. ? greedy not Hungarian; `score()` indexes the SAME preds/gts used to
build `match_result` or boundary errors are silently wrong.
107     - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
108     - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement`, `::evaluate`. ? `generalization_gap
>~0.1` = overfit alarm.
109     - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)` (?
hardcoded dup of real defaults), `::default_grid` (45 cfgs = 5x3x3), `::make_predict_fn`,
`::load_golden_gts` (? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ?
CLI defaults `fps=59.94/frame-width=1920` baked in.
110     - `@backend/eval/export_wasb_segments.py` ? trajectory+cfg -> golden/slicer CSV +
comparison. `::TUNED=(0.05,1.0,1.0)` (? hardcoded from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`. `--slice` requires `--video`.
111     - `@backend/eval/batch.py` ? corpus aggregation. `::aggregate` (micro pool TP/FP/FN, TP-
weighted mean_iou, macro F1), `::default_stages_for` (`auto`;
`::FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path`.
112     - `@backend/eval/harness.py` ? DB-pred scorer. `::STAGES=('candidates','final')`,
`::get_predictions` (candidates->`query_candidate_segments(detector=)`;
final->`query_play_segments({})` ALL segs, detector arg ignored),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate).
113     - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? assumes `row['stats']` is already a dict ? the DB layer
deserializes it; a raw JSON STR would AttributeError, not zero-fill; missing fields default
0.0), `::label_candidates` (same IoU matching as evaluator),
`::build_training_set`->(X,y,intervals).
114     - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
feature ORDER at predict must match fit (names are metadata only, not used in math); fixed 1000
epochs.
115     - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider`, `::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports
`annotation_registry` from `**`backend.annotations` NOT `backend.eval.annotations` (two
different "annotations"); gate fires on precision OR recall drop (F1 reported, not gated); first
run on a video silently passes (no baseline).
116
117     ### 2.4 Stitcher / tools / utils
118     - `@backend/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. $ skipped tuple
`(idx,start,end,reason)`. ? if duration probe fails (0.0) NO boundary validation; `padded_start`
uses `begin_padding*(0.5)/`end_padding*(1.0)` defaults; temp clips in a TemporaryDirectory under
output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses `print()`.
119     - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy,
kept in sync manually). `read_time` -> backend/eval/annotations.parse_timestamp.
120     - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe),
`::extract_video_chunk(...,reencode=False)`. ? uses `-t DURATION` (length, not end ts); copy
mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips.
121     - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (used by yolo_hybrid),
`::calculate_iou` (? +1 px convention inflates IoU), `::get_center/get_distance/get_velocity`.

```

```

122
123     ### 2.5 Validators ?
124     - `@backend/validators/base.py::VideoValidator` ABC, `::ValidationResult` (pydantic),
125       `::ValidationRegistry`, `::registry` (singleton). $ `validate(video_path, config, context)` ->
126       `ValidationResult`. ? **add a validator**: subclass + `registry.register(MyValidator())`
127       (registration order = execution order). ? validators share a per-run `context` dict; exceptions
128       wrapped as `passed=False` (never abort batch).
129     - `@backend/validators/__init__.py` ? registers in EXEC order: FormatValidator,
130       ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
131       RelevanceValidator (? producers before consumers ? format_check first).
132     - `@backend/validators/format.py::FormatValidator` (`format_check`) ? **PRODUCER** of
133       `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring
134       `mp4` (lowercased).
135     - `@backend/validators/resolution_fps.py::ResolutionFPSValidator`
136       (`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails if format_check didn't run
137       first). `min_fps` default 29.9; min res 1280x720.
138     - `@backend/validators/pts_continuity.py::PTSContinuityValidator`
139       (`pts_continuity_check`) ? ? 10.0s keyframe-gap threshold hardcoded, comment says 8.0s (drift);
140       any backward jump fails.
141     - `@backend/validators/scene_cut.py::SceneCutDensityValidator` (`scene_cut_check`) ? ?
142       `cut_threshold=0.6` hardcoded; samples at 2fps and caps at 100 SAMPLED frames (~first 50s of
143       video, not first 100 raw frames).
144     - `@backend/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
145       `min_blur_threshold`(100.0) = fail; samples 15 frames.
146     - `@backend/validators/relevance.py::RelevanceValidator` (`relevance_check`) ? HSV
147       court-green; lazy `import backend.sports`; ? unknown sport -> empty sport cfg, falls back to
148       defaults, does NOT fail.
149
150     ### 2.6 Sports / Providers / Annotations / DB ?
151     - `@backend/sports/base.py::SportProfile` ABC
152       (`name`, `get_prompt`, `get_relevance_config`);
153       `@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
154       `@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton). ?
155       **add a sport**: subclass `SportProfile`, `sport_registry.register(MyProfile)`. $ prompt emits
156       JSON array `{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int),
157       shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS
158       timestamps; `get()` returns a fresh instance each call.
159     - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
160       `@backend/providers/gemini.py::GeminiProvider` (upload->poll->generate JSON temp 0->delete);
161       `@backend/providers/__init__.py::provider_registry` (+`::ProviderRegistry`, auto-registers
162       `gemini`). ? **add a provider**: subclass, `provider_registry.register('name', MyCls)` (key
163       passed EXPLICITLY, unlike sport_registry); `get(name, api_key, model_name)`. $
164       `analyze_video(path, prompt, chunk_duration=0.0, label='')` -> (segments:list,
165       metrics:dict{upload_time, process_time, gen_time}). ? blocking `time.sleep(10)` poll; returns
166       `([], metrics)` on ANY failure (empty != error); lazy client (auth surfaces on first use).
167     - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`,
168       `::Interval` (= metrics.Interval), `::STATUSES`, `::AnnotationProviderRegistry`,
169       `::annotation_registry`. ? **add a tier**: subclass (define `name`), call
170       `annotation_registry.register(MyCls)` at module bottom + import in `__init__.py` (registry keys
171       off `provider_cls().name`). $ `fetch` -> List[Interval] sorted; contract says RAISE (not `[]`)
172       on unavailable (`[]` silently zeroes recall).
173     - `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the
174       resolved CSV path; -> `backend.eval.annotations.load_annotations`).
175     - `@backend/annotations/automated_provider.py::AutomatedProvider` (`auto`; injectable
176       `labeler`, `warn_if_same_lineage` ORACLE RULE, default `not_configured` RAISES
177       NotImplementedError). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`.
178     - `@backend/annotations/__init__.py` ? ? `noqa F401` imports are load-bearing (removing
179       de-registers providers).
180     - `@backend/database.py::Database` ? SQLite, 4 tables
181       (`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in
182       `__init_db` (currently =1). Methods: `add_video`, `add_play_segment`, `add_event`, `add_candidate_se
183       gment`, `link_candidate_to_segment`, `query_candidate_segments`, `query_play_segments`, `get_video`,
184       `clear_video_data`. ? every conn re-issues `PRAGMA foreign_keys=ON` (else CASCADE silently
185       skipped); `add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest;
186       write helpers swallow exceptions (return False/None); events column is `type` not `event_type`;

```

```

`duration` stored not derived.
141
142     ---
143
144     ## 3. DATA CONTRACTS (canonical shapes ? one place)
145
146     $ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
147     `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel
coords. Row = `(fid, 1|0, x or -1, y or -1)`.
148     -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
149
150     $ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`extract_action_windows_from_chunk`):
151     `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
152     `start/end = group_frame/fps + chunk_start`.
153
154     $ **candidate_segments DB row** (`db.add_candidate_segment` /
`query_candidate_segments`):
155     `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL,
duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
156     `detection_params` keys by detector:
wasb={detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}; tracknet
adds `queue_length`; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
157
158     $ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
159     `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
160
161     $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
162
163     $ **GT / golden CSVs** (TWO conventions ? do not conflate):
164     - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
165     - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
166
167     $ **det_raw.jsonl** (one window/line): `{"w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
168     $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
169     $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv,
...}]}`; ? DB key = file MD5, manifest `video_id` is a label.
170
171     $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
172
173     $ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
174
175     ---

```


176

```
177     ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
178     - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/detectors/wasb_infer.py::_build_cfg.)
179     - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
180     - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
hms_vis). `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
181     - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
182     - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
183     - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
184
185     ---
186
187     ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
188     - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
189     - `python -m src.cli.curate export` (collector) -> `
```

```

PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless
`format_check` ran earlier (shared `context['ffprobe_meta']`).
204 - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately.
205 - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both.
206 - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
207 - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
208 - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
209 - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat;
if probe fails (0.0) no boundary validation.
210
211 ---
212
213 ## 7. RAMP PATHS (to do X -> open Y::symbol)
214 | Task | Open |
215 |---|---|
216 | Tune candidate windowing (velocity/merge/min-dur) |
`@backend/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
217 | Add a segmenter | subclass `@backend/indexer/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in `@backend/indexer/__init__.py` |
218 | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
219 | Add a sport | subclass `@backend/sports/base.py::SportProfile`;
`sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
220 | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', cls)` |
221 | Add an annotation/GT tier | subclass
`@backend/annotations/base.py::AnnotationProvider`; register + import in
`@backend/annotations/__init__.py` |
222 | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main`
(`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |
223 | Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main`
(`--slice` needs `--video`) |
224 | Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
225 | Score DB preds vs GT (one video) | `@run_eval.py::main` |
226 | Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
227 | Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress,
2=missing DB; snapshot via `--update-baseline` |
228 | Resume a crashed WASB pass | nothing to do ? `@backend/detectors/wasb_infer.py::run`
auto-resumes from `det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart;
cache invalidates on weights/sport/frames change |
229 | Add a validator | subclass `@backend/validators/base.py::VideoValidator`;
`registry.register(instance)` in `@backend/validators/__init__.py` (mind order; producers before
consumers) |
230 | Add a DB column/table | `@backend/database.py::Database._init_db` migration ladder +
bump `PRAGMA user_version` (else version-assert tests fail) |
231 | Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
232
233 ---
234
235 ## 8. TEST MAP (`tests/test_*.py` -> subsystem)
236 | Test | Covers |
237 |---|---|
238 | `test_base.py` | `indexer.base.SegmenterRegistry` (register/get/list via
DummySegmenter) |
239 | `test_yolo_hybrid.py` / `test_tracknet_hybrid.py` | hybrid segmenters (yolo
torchvision+ByteTrack; tracknet 4-phase) |

```

```

240 | `test_gemini.py` | `indexer.gemini.GeminiPlaySegmenter` (parse/validate/chunk) |
241 | `test_tracknet_runner.py` | `detectors.tracknet_runner` parse_trajectory_csv +
trajectory_to_action_windows (shared brain) |
242 | `test_wasb_cache.py` | `detectors.wasb_infer` resumable cache (serialize, contiguity
heal, manifest invalidation, atomic writes) |
243 | `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
244 | `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate |
245 | `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict |
246 | `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate |
247 | `test_calibration.py` | `eval.calibration` kfold/select_best/cross_validate/LOVO |
248 | `test_calibrate_wasb.py` | `eval.calibrate_wasb` CLI/run/grid |
249 | `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment+comparison export
|
250 | `test_eval_training.py` | `eval.training` feature extraction + labeling |
251 | `test_eval_classifier.py` | `eval.classifier.LogisticRegression` fit/predict/save/load
|
252 | `test_eval_regression.py` | `eval.regression` baseline compare/gate |
253 | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label
parse |
254 | `test_video.py` | `utils.video` duration/chunk |
255 | `test_geometry.py` | `utils.geometry` center/distance/velocity/IoU |
256 | `test_database.py` | `database.Database` schema/CRUD/migrations/version |
257 | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile |
258 | `test_providers.py` / `test_provider_gemini.py` / `test_gemini.py` | `providers`
registry + GeminiProvider lifecycle |
259 | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
registry, FileProvider, AutomatedProvider (oracle/lineage) |
260 | (no dedicated test) | `validators/*`, `stitcher.compiler`, `main.py`, `run*.py`
orchestrators, `wasb_runner`, `wasb_hybrid`/`tracknet_hybrid` (covered indirectly) ?? exercised
only indirectly |
261

```

4. user (2026-06-07T06:51:40.618Z)

```

1     from abc import ABC, abstractmethod
2     from pydantic import BaseModel
3     from typing import List, Dict, Any, Optional
4
5     class ValidationResult(BaseModel):
6         validator_name: str
7         passed: bool
8         message: str
9         details: Dict[str, Any] = {}
10
11     class VideoValidator(ABC):
12         @property
13         @abstractmethod
14         def name(self) -> str:
15             """Unique identifier for this validator."""
16             pass
17
18         @property
19         @abstractmethod
20         def description(self) -> str:
21             """Human-readable explanation of what this validator checks."""
22             pass
23
24         @abstractmethod
25         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
26             """
27             Executes validation.
28             - video_path: Local path to the MP4 file.
29             - config: Dict parsed from config.json.
30             - context: Dynamic context dict shared across validators (stores shared values

```

```

like ffprobe metadata).
31         """
32         pass
33
34     class ValidationRegistry:
35         def __init__(self):
36             self._validators: List[VideoValidator] = []
37
38         def register(self, validator: VideoValidator):
39             """Registers a new validator."""
40             self._validators.append(validator)
41
42         def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
43             """Runs all registered validators in sequence, accumulating results."""
44             results = []
45             context = {} # Shared context dictionary (e.g. for caching ffprobe outputs)
46             for val in self._validators:
47                 try:
48                     res = val.validate(video_path, config, context)
49                     results.append(res)
50                 except Exception as e:
51                     results.append(ValidationResult(
52                         validator_name=val.name,
53                         passed=False,
54                         message=f"Validator threw an unexpected error: {str(e)}"
55                     ))
56             return results
57
58     # Shared global registry
59     registry = ValidationRegistry()
60

```

5. assistant (2026-06-07T06:51:51.860Z)

I have a strong starting point from the CODE_MAP. Let me verify the key integration points by reading the actual orchestration code in both repos, plus the existing collector report pattern that we must reuse.

6. user (2026-06-07T06:51:53.245Z)

```

80         segment_ids: List[int]
81         output_filename: str
82
83     # --- API Endpoints ---
84     @app.get("/api/config")
85     def get_config():
86         return global_config
87
88     @app.post("/api/process")
89     def process_video(req: ProcessRequest):
90         if not os.path.exists(req.video_path):
91             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
92
93         video_id = get_file_md5(req.video_path)
94
95         # 1. Run pluggable validators
96         print(f"[API] Running validations for {req.video_path}...")
97         val_results = registry.run_all(req.video_path, global_config)
98
99         # Check if any validator failed
100         failures = [r for r in val_results if not r.passed]
101         if failures:
102             # Save failed processing log

```

```

103         db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
104         return {
105             "video_id": video_id,
106             "status": "failed",
107             "message": "Video failed validation checks.",
108             "results": [r.dict() for r in val_results]
109         }
110
111     # 2. Run segmenter & indexer
112     print(f"[API] Video passed checks. Segmenting and indexing...")
113     db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
114
115     seg_name = req.segmenter or global_config.get("indexing",
{ }).get("default_segmenter", "motion")
116     segmenter_cls = segmenter_registry.get(seg_name)
117     if not segmenter_cls:
118         available = list(segmenter_registry.list_available().keys())
119         raise HTTPException(
120             status_code=400,
121             detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
122         )
123
124     print(f"[API] Using segmenter: {seg_name}")
125     segmenter = segmenter_cls(db_instance, global_config)
126     segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
127
128     return {
129         "video_id": video_id,
130         "status": "success",
131         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
132         "results": [r.dict() for r in val_results],
133         "play_segments": segments
134     }
135
136 @app.post("/api/query")
137 def query_play_segments(req: QueryRequest):
138     filters = {
139         "server": req.server,
140         "receiver": req.receiver,
141         "duration_min": req.duration_min,
142         "event_type": req.event_type
143     }
144     segments = db_instance.query_play_segments(req.video_id, filters)
145     return {"play_segments": segments}
146
147 @app.post("/api/compile")
148 def compile_highlights(req: CompileRequest):
149     video = db_instance.get_video(req.video_id)
150     if not video:
151         raise HTTPException(status_code=404, detail="Indexed video record not found.")
152
153     # Retrieve matching play segments from db
154     all_segments = db_instance.query_play_segments(req.video_id, { })
155     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
156
157     if not matched_segments:
158         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
159

```

7. user (2026-06-07T06:51:53.687Z)

```
300     global global_config, db_instance
301     global_config = load_config()
302     # Merge output_dir into config
303     if "output" not in global_config:
304         global_config["output"] = {}
305     global_config["output"]["output_dir"] = args.output_dir
306     os.makedirs(args.output_dir, exist_ok=True)
307
308     db_path = os.path.join(args.output_dir, global_config.get("output",
309 {}).get("db_name", "sports_indexer.db"))
310     db_instance = Database(db_path)
311
312     # CLI processing mode (no web UI needed)
313     if args.video_path and not args.server:
314         print(f"[CLI] Processing video file: {args.video_path}")
315         if not os.path.exists(args.video_path):
316             print(f"[FAIL] Video file not found: {args.video_path}")
317             return
318
319         video_id = get_file_md5(args.video_path)
320
321         # Validate
322         print("[CLI] Validating...")
323         val_results = registry.run_all(args.video_path, global_config)
324         failures = [r for r in val_results if not r.passed]
325
326         # Save video status
327         status = "failed" if failures else "processed"
328         db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
329 val_results])
330
331         print("\n" + "="*40)
332         print("VALIDATION SUMMARY")
333         print("="*40)
334         for r in val_results:
335             status_symbol = "[PASS]" if r.passed else "[FAIL]"
336             print(f"{status_symbol} {r.validator_name}: {r.message}")
337         print("="*40 + "\n")
338
339         if failures:
340             print("[FAIL] Video failed checks. Indexing halted.")
341             return
342
343         # Index
344         seg_name = args.segmenter or global_config.get("indexing",
345 {}).get("default_segmenter", "motion")
346         segmenter_cls = segmenter_registry.get(seg_name)
347         if not segmenter_cls:
348             print(f"[FAIL] Segmenter '{seg_name}' not found.")
349             return
350
351         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
352         segmenter = segmenter_cls(db_instance, global_config)
353         segments = segmenter.process_video(video_id, args.video_path,
354 max_frames=args.max_frames)
355         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
356 database: {db_path}")
357         return
358
359     # Start Local Server Mode
360     if args.server or not args.video_path:
361         print(f"[SERVER] Starting local FastAPI server on
362 http://localhost:{args.port}...")
```

```

357         uvicorn.run(app, host="127.0.0.1", port=args.port)
358
359     if __name__ == "__main__":

```

8. assistant (2026-06-07T06:51:56.361Z)

Confirmed the indexer hooks. Now let me examine the collector's existing report pattern and CLI, which we must reuse rather than duplicate.

9. user (2026-06-07T06:51:57.054Z)

```

1     """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3     Why this exists
4     -----
5     Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6     per-rally CSV that imports directly via curate import-local. A vendor may
7     instead do the work in CVAT and hand back the CVAT export: per-frame bounding
8     boxes, with the rally fields (rally_number, shots_count,
9     ending_reason ...) carried as CVAT object attributes on each box. The box
10    geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11    metadata to.
12
13    This module is the ingestion-side adapter that absorbs that vendor format so the
14    modeling repo never has to. It does two things, both deliberately on our side
15    so we fix mechanical problems rather than renegotiating them:
16
17    1. Recompute time from the frame range. A rally's interval is taken from the
18    first/last frame a box for that rally appears on:
19    seconds = frame / fps at the video's true fps. We ignore any
20    start_time/end_time attributes the vendor wrote ? a CVAT job that
21    samples every step frames and assumes 30 fps yields systematically wrong
22    times (we have seen start_time == frame / (step * 30), i.e. off by the
23    sampling step at the wrong fps). The frame index is reliable; the vendor's
24    seconds are not.
25
26    2. Surface ? never silently fix ? content defects. Overlapping rallies,
27    zero-duration rallies, ending_reason values outside the agreed set, and
28    too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29    rally boundary would poison ground truth, so that stays a human/vendor call.
30
31    Supported input: the single-file CVAT XML export, in either image mode
32    (<image><box>...) or track/interpolation mode (<track><box frame=...>).
33    The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34    later if a vendor sends only those.
35    """
36    from __future__ import annotations
37
38    import os
39    import xml.etree.ElementTree as ET
40    from collections import Counter
41    from dataclasses import dataclass, field
42    from typing import Any, Dict, List, Optional, Tuple
43
44    from src.core.parsing import safe_int
45    from src.core.downloader import probe_resolution_fps
46
47    # The agreed cross-sport ending_reason vocabulary for net-separated sports.
48    # Kept here as a soft check only (a warning, never a hard failure) so this
49    # adapter does not depend on the annotation-spec wording.
50    DEFAULT_ALLOWED_REASONS = {
51        "winner",
52        "forced_error",
53        "unforced_error",
54        "service_fault",

```

```

55         "let",
56         "other",
57     }
58
59     # The observational outcome of the last shot ? orthogonal to ending_reason
60     # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
61     DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
62
63     # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
64     # schema has used suffixed names like "score_before (A-B)"; we match leniently.
65     _ATTR_ALIASES = {
66         "rally_number": ("rally_number",),
67         "shots_count": ("shots_count",),
68         "ending_reason": ("ending_reason",),
69         "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
70         "score_before": ("score_before", "score_before (a-b)"),
71         "score_after": ("score_after", "score_after (a-b)"),
72         "notes": ("notes", "note"),
73     }
74
75     CANONICAL_COLUMNS = [
76         "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
77         "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
"notes",
78     ]
79
80
81     @dataclass
82     class RallyRecord:
83         """One rally recovered from the CVAT export, time re-derived from frames."""
84         rally_number: int
85         start_time: float          # seconds, = frame_min / fps
86         end_time: float            # seconds, = frame_max / fps (see padding note)
87         shots_count: Optional[int]
88         ending_reason: Optional[str]
89         last_shot_outcome: Optional[str]
90         score_before: Optional[str]
91         score_after: Optional[str]
92         notes: Optional[str]
93         frame_min: int
94         frame_max: int
95         n_boxes: int
96         vendor_start_time: Optional[str] = None    # the (discarded) attribute value
97         vendor_end_time: Optional[str] = None
98
99         @property
100         def duration(self) -> float:
101             return self.end_time - self.start_time
102
103
104     @dataclass
105     class Issue:
106         """A defect found in the delivery. ``blocker`` issues stop a clean import."""
107         severity: str    # 'blocker' | 'high' | 'medium' | 'low'
108         code: str
109         message: str
110         rally_number: Optional[int] = None
111
112
113     @dataclass
114     class ConversionResult:
115         records: List[RallyRecord]
116         issues: List[Issue]
117         fps: float

```



```

118     step: int
119     meta: Dict[str, Any] = field(default_factory=dict)
120
121     @property
122     def n_blockers(self) -> int:
123         return sum(1 for i in self.issues if i.severity == "blocker")
124
125
126 def _mmss(seconds: float) -> str:
127     """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
128     m = int(seconds // 60)
129     s = seconds - m * 60
130     return f"{m}:{s:06.3f}"
131
132
133 def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
134     for alias in _ATTR_ALIASES[canonical]:
135         if alias in attrs and attrs[alias] != "":
136             return attrs[alias]
137     return None
138
139
140 def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
141     """Parse a CVAT XML export into (meta, boxes).
142
143     ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
144     entry per visible box, from either image-mode or track-mode exports.
145     ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
146     """
147     if not os.path.exists(xml_path):
148         raise FileNotFoundError(f"CVAT export not found: {xml_path}")
149
150     root = ET.parse(xml_path).getroot()
151     meta: Dict[str, Any] = {}
152
153     job = root.find("./job")
154     if job is not None:
155         stop = job.findtext("stop_frame")
156         if stop is not None and stop.strip().isdigit():
157             meta["stop_frame"] = int(stop.strip())
158         ff = job.findtext("frame_filter") or ""
159         if "step=" in ff:
160             try:
161                 meta["step"] = int(ff.split("step=")[1].split()[0])
162             except (ValueError, IndexError):
163                 pass
164
165     def _attrs(el: ET.Element) -> Dict[str, str]:
166         out: Dict[str, str] = {}
167         for a in el.findall("attribute"):
168             out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
169         return out
170
171     boxes: List[Dict[str, Any]] = []
172
173     # Image mode: <image id=frame> ... <box>...</box>
174     for img in root.findall("image"):
175         try:
176             frame = int(img.get("id"))
177         except (TypeError, ValueError):
178             continue
179         for box in img.findall("box"):
180             boxes.append({"frame": frame, "attrs": _attrs(box)})
181
182     # Track mode: <track> ... <box frame=N outside=0>...</box>

```

```

183     for track in root.findall("track"):
184         for box in track.findall("box"):
185             if box.get("outside") == "1":
186                 continue # a track's terminating keyframe carries no real label
187             try:
188                 frame = int(box.get("frame"))
189             except (TypeError, ValueError):
190                 continue
191             boxes.append({"frame": frame, "attrs": _attrs(box)})
192
193     return meta, boxes
194
195
196 def _infer_step(frames: List[int]) -> int:
197     """Most common positive gap between consecutive distinct sampled frames."""
198     uniq = sorted(set(frames))
199     gaps = [b - a for a, b in zip(uniq, uniq[1:]) if b - a > 0]
200     if not gaps:
201         return 1
202     return Counter(gaps).most_common(1)[0][0]
203
204
205 def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
206     """Group boxes by rally_number and derive each rally's frame range -> seconds."""
207     if fps is None or fps <= 0:
208         raise ValueError("fps must be a positive number to convert frames to seconds")
209
210     by_rally: Dict[int, List[Dict[str, Any]]] = {}
211     for b in boxes:
212         rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
213         if rn is None:
214             continue
215         by_rally.setdefault(rn, []).append(b)
216
217     records: List[RallyRecord] = []
218     for rn in sorted(by_rally):
219         items = by_rally[rn]
220         frames = sorted(b["frame"] for b in items)
221         attrs = items[0]["attrs"]
222         fmin, fmax = frames[0], frames[-1]
223         records.append(RallyRecord(
224             rally_number=rn,
225             start_time=fmin / fps,
226             end_time=fmax / fps,
227             shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
228             ending_reason=_attr_lookup(attrs, "ending_reason"),
229             last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
230             score_before=_attr_lookup(attrs, "score_before"),
231             score_after=_attr_lookup(attrs, "score_after"),
232             notes=_attr_lookup(attrs, "notes"),
233             frame_min=fmin,
234             frame_max=fmax,
235             n_boxes=len(frames),
236             vendor_start_time=attrs.get("start_time"),
237             vendor_end_time=attrs.get("end_time"),
238         ))
239     return records
240
241
242 def detect_issues(records: List[RallyRecord],
243                  stop_frame: Optional[int] = None,
244                  allowed_reasons: Optional[set] = None,
245                  allowed_outcomes: Optional[set] = None,
246                  max_gap_s: float = 15.0) -> List[Issue]:
247     """Find the defects that block or degrade a clean ground-truth import.

```

```

248
249 Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
250 slack of ~0.5s is acceptable for this pipeline because the downstream
251 highlight compiler pads every clip on output (badminton-highlight-indexer,
252 backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
253 absorbs the sampling uncertainty when highlights are trimmed.
254 """
255 allowed = allowed_reasons if allowed_reasons is not None else
DEFAULT_ALLOWED_REASONS
256 allowed_out = allowed_outcomes if allowed_outcomes is not None else
DEFAULT_ALLOWED_OUTCOMES
257 issues: List[Issue] = []
258
259 ordered = sorted(records, key=lambda r: r.rally_number)
260 for i, r in enumerate(ordered):
261     if r.end_time <= r.start_time:
262         issues.append(Issue(
263             "blocker", "zero_duration",
264             f"Rally {r.rally_number}: end <= start "
265             f"({r.frame_min}-{r.frame_max}); single-sample or stray box.",
266             r.rally_number))
267
268     if r.ending_reason is not None and r.ending_reason not in allowed:
269         issues.append(Issue(
270             "medium", "ending_reason_vocab",
271             f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
272             f"not in the agreed set {sorted(allowed)}.",
273             r.rally_number))
274
275     if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
276         issues.append(Issue(
277             "low", "outcome_vocab",
278             f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
279             f"not in {sorted(allowed_out)}.",
280             r.rally_number))
281
282     # Inflated span: a long rally with implausibly few shots usually means a
283     # stray/duplicated box stretched the frame range past the real ending.
284     if r.shots_count and r.duration >= 5.0:
285         sps = r.shots_count / r.duration
286         if sps < 0.3:
287             issues.append(Issue(
288                 "medium", "shot_density",
289                 f"Rally {r.rally_number}: {r.shots_count} shots over "
290                 f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
291                 f"may be inflated by a stray box.",
292                 r.rally_number))
293
294     if stop_frame is not None and r.frame_max > stop_frame:
295         issues.append(Issue(
296             "high", "frame_out_of_range",
297             f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
298             f"last frame {stop_frame}.",
299             r.rally_number))
300
301     if i > 0:
302         prev = ordered[i - 1]
303         if r.start_time < prev.end_time:
304             issues.append(Issue(
305                 "blocker", "overlap",
306                 f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
307                 f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
308                 f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
309                 r.rally_number))
310         elif r.start_time - prev.end_time > max_gap_s:

```

```

311         # A long unlabeled stretch between two rallies is a cheap, label-only
312         # second opinion for the worst error (a missed rally). It won't catch a
313         # rally MERGED into its neighbour's span (shot_density/inflated catches
314         # that), but it does catch an omitted rally that left a visible hole.
315         gap = r.start_time - prev.end_time
316         issues.append(Issue(
317             "medium", "possible_missed_rally",
318             f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
319             f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
320             f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
gap.",
321             r.rally_number))
322     return issues
323
324
325 def convert(xml_path: str,
326            fps: Optional[float] = None,
327            video_path: Optional[str] = None,
328            step: Optional[int] = None,
329            allowed_reasons: Optional[set] = None) -> ConversionResult:
330     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
331
332     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
333     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
334     that silent guess is exactly the bug this adapter exists to undo.
335     """
336     meta, boxes = parse_cvat_boxes(xml_path)
337
338     if fps is None and video_path:
339         _, _, probed = probe_resolution_fps(video_path)
340         fps = probed
341     if fps is None or fps <= 0:
342         raise ValueError(
343             "Could not determine fps. Pass fps=... or a readable video_path so "
344             "frames can be converted to seconds (seconds = frame / fps).")
345
346     if step is None:
347         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
348
349     records = build_records(boxes, fps)
350     issues = detect_issues(records,
351                            stop_frame=meta.get("stop_frame"),
352                            allowed_reasons=allowed_reasons)
353     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
meta=meta)
354
355
356 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
357     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
358     import csv
359     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
360     with open(out_path, "w", newline="", encoding="utf-8") as f:
361         w = csv.writer(f)
362         w.writerow(CANONICAL_COLUMNS)
363         for r in sorted(records, key=lambda x: x.rally_number):
364             w.writerow([
365                 r.rally_number,
366                 _mmss(r.start_time), _mmss(r.end_time),
367                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
368                 r.shots_count if r.shots_count is not None else "",
369                 r.ending_reason or "",
370                 r.last_shot_outcome or "",
371                 r.score_before or "", r.score_after or "", r.notes or "",
372             ])
373

```

```

374
375 def render_issue_report(result: ConversionResult, source: str) -> str:
376     """A human-readable issue report (for review / sending back to the vendor)."""
377     lines = [
378         f"CVAT rally conversion report ? {source}",
379         f"  fps used          : {result.fps:.3f}",
380         f"  sampling step      : {result.step} frames (~{result.step / result.fps:.2f}s)",
381         f"  rallies parsed     : {len(result.records)}",
382         f"  issues              : {len(result.issues)} ({result.n_blockers} blocker)",
383         "",
384     ]
385     if not result.issues:
386         lines.append("No issues detected.")
387     else:
388         for sev in ("blocker", "high", "medium", "low"):
389             group = [i for i in result.issues if i.severity == sev]
390             if not group:
391                 continue
392             lines.append(f"[{sev.upper()}]")
393             for i in group:
394                 lines.append(f"  - ({i.code}) {i.message}")
395             lines.append("")
396     return "\n".join(lines)
397
398
399 # ----- #
400 # Validation reporting ? used by `curate validate-delivery`. #
401 # These turn a (records, issues) pair into the artifacts a human reviews: #
402 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
403 # ----- #
404
405 def _parse_secs(value: Optional[str]) -> Optional[float]:
406     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
407     s = (value or "").strip()
408     if not s:
409         return None
410     if ":" in s:
411         try:
412             total = 0.0
413             for part in s.split(":"):
414                 total = total * 60 + float(part)
415             return total
416         except ValueError:
417             return None
418     try:
419         return float(s)
420     except ValueError:
421         return None
422
423
424 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
425     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
426     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
427     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
428     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
429     """
430     import csv as _csv
431     if not os.path.exists(csv_path):
432         raise FileNotFoundError(f"CSV not found: {csv_path}")
433     records: List[RallyRecord] = []
434     with open(csv_path, newline="", encoding="utf-8") as f:
435         reader = _csv.DictReader(f)
436         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
437 ]]
438         for raw in reader:

```

```

438         row = {(k or "").strip().lower(): v for k, v in row.items()}
439         rn = safe_int(row.get("rally_number"), None)
440         if rn is None:
441             continue
442         start = _parse_secs(row.get("start_time")) or
_parse_secs(row.get("start_mmss"))
443         end = _parse_secs(row.get("end_time")) or _parse_secs(row.get("end_mmss"))
444         if start is None or end is None:
445             continue
446         records.append(RallyRecord(
447             rally_number=rn, start_time=start, end_time=end,
448             shots_count=safe_int(row.get("shots_count"), None),
449             ending_reason=(row.get("ending_reason") or "").strip() or None,
450             last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
451             score_before=(row.get("score_before") or "").strip() or None,
452             score_after=(row.get("score_after") or "").strip() or None,
453             notes=(row.get("notes") or "").strip() or None,
454             frame_min=-1, frame_max=-1, n_boxes=0,
455         ))
456     return records
457
458
459 def build_summary(records: List[RallyRecord], issues: List[Issue],
460                  source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
461     """Machine-readable validation summary (JSON / CI)."""
462     by_sev = {s: [i for i in issues if i.severity == s]
463              for s in ("blocker", "high", "medium", "low")}
464     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
465     return {
466         "source": source,
467         "fps": fps,
468         "n_rallies": len(records),
469         "n_issues": len(issues),
470         "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
471         "n_blockers": len(by_sev["blocker"]),
472         "passed": len(by_sev["blocker"]) == 0,
473         "flagged_rallies": flagged,
474         "issues": [{"severity": i.severity, "code": i.code,
475                    "rally_number": i.rally_number, "message": i.message} for i in
476         issues],
477     }
478
479 def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
-> str:
480     """Human validation report: scoreboard + issues by severity + rallies to review."""
481     n = len(records)
482     by_sev = {s: [i for i in issues if i.severity == s]
483              for s in ("blocker", "high", "medium", "low")}
484     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
485     clean = n - len(flagged)
486     lines = [
487         f"# Delivery validation ? {source}",
488         "",
489         f"- rallies: **{n}**",
490         f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
491         f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
492         {len(by_sev['low'])})",
493         f"- structurally-clean rallies: **{clean}/{n}**"
494         + (f" ({100 * clean // n}%" if n else ""),
495         f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
496         "",
497     ]
498     if issues:
499         for sev in ("blocker", "high", "medium", "low"):

```

```

499         grp = by_sev[sev]
500         if not grp:
501             continue
502         lines.append(f"## {sev.upper()}")
503         lines += [f"- ({i.code}) {i.message}" for i in grp]
504         lines.append("")
505     else:
506         lines.append("No issues detected. ?\n")
507     if flagged:
508         lines.append(f"## Rallies to review: {flagged}")
509     return "\n".join(lines)
510
511
512 def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
None:
513     """Auto-generate the annotatable manual-verification sheet (blank `true_*` cols)."""
514     import csv
515     flags: Dict[int, List[str]] = {}
516     for i in issues:
517         if i.rally_number is not None:
518             flags.setdefault(i.rally_number, []).append(i.code)
519     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
520     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
521            "ending_reason", "last_shot_outcome", "auto_flags",
522            "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
523            "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
524     with open(out_path, "w", newline="", encoding="utf-8") as f:
525         w = csv.writer(f)
526         w.writerow(cols)
527         for r in sorted(records, key=lambda x: x.rally_number):
528             w.writerow([
529                 r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
530                 f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
531                 r.ending_reason or "", r.last_shot_outcome or "",
532                 ";".join(flags.get(r.rally_number, [])),
533                 "", "", "", "", "", "",
534             ])
535

```

10. assistant (2026-06-07T06:52:01.151Z)

This is a well-structured existing report module. Now let me look at the collector CLI orchestration to understand the per-stage hooks, and check the existing FAQ/README and ADR-005 in the indexer.

11. user (2026-06-07T06:52:02.523Z)

```

420         print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _,a,_ in
results if a=='have')}}    "
421         f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}}    Failed :
{len(failed)}")
422     if failed:
423         print("Failures (left unchanged in DB):")
424         for vid, _, detail in failed:
425             print(f"  - {vid}: {detail}")
426     print()
427
428 def export_eval_set(self, args):
429     """Export curated annotations as indexer-ready eval/training sets.
430
431     Emits, per qualifying video, a `/sport/<video_id>.csv` in the
432     indexer's `start,end` ground-truth format (decimal seconds), plus a single
433     `/manifest.json` that pairs each CSV with its video file. The
434     manifest is the contract the downstream eval/training driver reads: the
435     indexer keys videos by `file MD5`, not by our `video_id`, so it must run

```

```

436         against the exact `local_path`/`video_url` recorded here.
437
438     By default only CURATED + commercially-safe videos are exported, mirroring
439     the license gate already enforced elsewhere in the toolchain.
440     """
441     out_dir = args.out_dir
442     commercial_only = not args.include_noncommercial
443     statuses = [CurationStatus.CURATED]
444     if args.include_staging:
445         statuses.append(CurationStatus.STAGING)
446
447     supported = self.config.get("supported_sports", ["badminton"])
448     sport_strs = [args.sport] if args.sport else supported
449
450     manifest = []
451     skipped_noncomm = 0
452     skipped_empty = 0
453
454     for sport_str in sport_strs:
455         try:
456             sport = SportType(sport_str.lower())
457         except ValueError:
458             print(f"Warning: skipping unknown sport '{sport_str}'.")
459             continue
460
461         videos = []
462         for status in statuses:
463             videos.extend(self.db.get_videos_by_status(status, sport))
464
465         for video in videos:
466             if commercial_only and not video.is_commercial:
467                 skipped_noncomm += 1
468                 continue
469
470             intervals = self.db.get_segment_intervals(video.video_id, sport)
471             if not intervals:
472                 skipped_empty += 1
473                 continue
474
475             sport_dir = os.path.join(out_dir, sport.value)
476             os.makedirs(sport_dir, exist_ok=True)
477             csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
478             with open(csv_path, "w", newline="", encoding="utf-8") as f:
479                 writer = csv.writer(f)
480                 writer.writerow(["start", "end"])
481                 for start, end in intervals:
482                     writer.writerow([start, end])
483
484             manifest.append({
485                 "video_id": video.video_id,
486                 "sport": sport.value,
487                 "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
488                 "local_path": video.local_path,
489                 "video_url": video.video_url,
490                 "match_name": video.match_name,
491                 "fps": video.fps,
492                 "resolution": video.resolution,
493                 "license_type": video.license_type.value,
494                 "is_commercial": video.is_commercial,
495                 "status": video.status.value,
496                 "num_segments": len(intervals),
497             })
498
499     os.makedirs(out_dir, exist_ok=True)
500     manifest_doc = {

```



```

501         "commercial_only": commercial_only,
502         "statuses": [s.value for s in statuses],
503         "total_videos": len(manifest),
504         "total_segments": sum(m["num_segments"] for m in manifest),
505         "eval_sets": manifest,
506     }
507     manifest_path = os.path.join(out_dir, "manifest.json")
508     with open(manifest_path, "w", encoding="utf-8") as f:
509         json.dump(manifest_doc, f, indent=2)
510
511     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
512     if manifest:
513         summary = [[m["sport"], m["video_id"], m["num_segments"],
514                     "YES" if m["is_commercial"] else "NO",
515                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
516                    for m in manifest]
517         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
518         print(f"\nVideos exported : {len(manifest)}")
519         print(f"Total segments : {manifest_doc['total_segments']}")
520         if skipped_noncomm:
521             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
522         if skipped_empty:
523             print(f"Skipped (no annotations) : {skipped_empty}")
524         print(f"Manifest : {manifest_path}\n")
525
526     def convert_cvat(self, args):
527         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
528
529         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
530         attributes) on the ingestion side: each rally's time is recomputed from
531         its frame range at the video's true fps, and content defects (overlaps,
532         zero-duration, off-vocabulary endings, coarse sampling) are written to an
533         issues report rather than silently "fixed". The emitted CSV imports via
534         `import-local`; any blocker issue must be resolved first, since the
535         importer's validator rejects overlapping / zero-duration rallies.
536         """
537         from src.parsers.cvat.rally_cvat import (
538             convert, write_canonical_csv, render_issue_report,
539         )
540         try:
541             result = convert(args.input, fps=args.fps,
542                             video_path=args.video_path, step=args.step)
543         except (FileNotFoundError, ValueError) as e:
544             print(f"Error: {e}")
545             sys.exit(1)
546
547         write_canonical_csv(result.records, args.out)
548         report = render_issue_report(result, os.path.basename(args.input))
549         if args.issues:
550             with open(args.issues, "w", encoding="utf-8") as f:
551                 f.write(report)
552         print(report)
553         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
554         if result.n_blockers:
555             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
556
557                 f"this file until they are resolved (by the vendor or manual
review).")
558         else:
559             print("\n No blockers. Next:")
560             print(f"    python -m src.cli.curate import-local --sport <sport> "
f"--video-path <video> --annotations-path {args.out} --fps

```

```

{result.fps:g}")
561
562     def validate_delivery(self, args):
563         """Run the full validation suite on a rally delivery and emit a report.
564
565         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
566         CSV (--csv). Writes, into --out-dir:
567             <id>_report.md    human report (scoreboard + issues + rallies to review)
568             <id>_report.json  machine summary (for CI / a bulk-ingest loop)
569             <id>_review.csv   the annotatable manual-verification sheet
570         Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
571         """
572         import json
573         from src.parsers.cvat.rally_cvat import (
574             convert, load_canonical_csv, detect_issues,
575             build_summary, render_validation_md, write_review_csv,
576         )
577         vid = args.video_id or "delivery"
578         fps = None
579         if args.input:
580             try:
581                 result = convert(args.input, fps=args.fps, video_path=args.video_path)
582             except (FileNotFoundError, ValueError) as e:
583                 print(f"Error: {e}")
584                 sys.exit(1)
585             records, issues, fps = result.records, result.issues, result.fps
586         elif args.csv:
587             try:
588                 records = load_canonical_csv(args.csv)
589             except FileNotFoundError as e:
590                 print(f"Error: {e}")
591                 sys.exit(1)
592             if not records:
593                 print(f"Error: no usable rally rows in {args.csv}")
594                 sys.exit(1)
595             issues = detect_issues(records)
596         else:
597             print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
598             sys.exit(1)
599
600         os.makedirs(args.out_dir, exist_ok=True)
601         summary = build_summary(records, issues, source=vid, fps=fps)
602         md = render_validation_md(records, issues, source=vid)
603         json_path = os.path.join(args.out_dir, f"{vid}_report.json")
604         md_path = os.path.join(args.out_dir, f"{vid}_report.md")
605         review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
606         with open(json_path, "w", encoding="utf-8") as f:
607             json.dump(summary, f, indent=2)
608         with open(md_path, "w", encoding="utf-8") as f:
609             f.write(md)
610         write_review_csv(records, issues, review_path)
611
612         print(md)
613         print(f"\nWrote:\n {md_path}\n {json_path}\n {review_path}")
614         if summary["n_blockers"]:
615             print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
616             sys.exit(2)
617         print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
618
619     def curate(self):
620         """Interactive loop to review and curate staging video annotations."""
621         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
622         if not staging_videos:
623             print("\nNo staging video annotations to review.\n")

```

```

624         return
625
626         print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
627         table_data = []
628         for idx, video in enumerate(staging_videos):
629             table_data.append([

```

12. user (2026-06-07T06:52:02.985Z)

```

1         """Best-quality video acquisition.
2
3         A single place that knows how to fetch the *highest resolution available* for a
4         source URL and report what was actually obtained. Both the ShuttleSet crawler
5         (initial ingestion) and the `fetch` CLI command (in-place upgrade of already-
6         ingested videos) go through here, so the resolution policy lives in exactly one
7         spot.
8
9         Design notes:
10        - `bv*+ba/b` selects best video + best audio, falling back to the best
11          pre-muxed stream. Combined with `-S res,fps,...`, yt-dlp sorts candidate
12          formats by resolution (then fps) descending and picks the top ? i.e. *always*
13          the best resolution available, regardless of codec/container quirks.
14        - `--merge-output-format mp4` guarantees an MP4 container, which the downstream
15          indexer requires.
16        - We never trust the requested resolution; we ffprobe the resulting file and
17          record the *actual* width/height/fps as the source of truth.
18        """
19
20        import os
21        import shutil
22        import subprocess
23        from dataclasses import dataclass
24        from typing import List, Optional, Tuple
25
26        # Best video+audio, prefer mp4/m4a, fall back to best single stream.
27        _FORMAT = "bv*+ba/b"
28        # Sort candidates so the highest resolution (then fps) wins; prefer mp4/m4a on ties.
29        _SORT = "res,fps,ext:mp4:m4a"
30
31
32        def height_to_label(height: Optional[int]) -> Optional[str]:
33            """Map a pixel height to a conventional resolution label (e.g. 1080 -> '1080p').
34
35            Snaps to the nearest standard tier at or below the actual height so a 1072px
36            crop still reads as '1080p' rather than an odd number.
37            """
38            if not height or height <= 0:
39                return None
40            # Allow a 5% tolerance below each tier so a lightly letterboxed/cropped frame
41            # (e.g. 1072px) still reads as the intended tier ('1080p') rather than dropping.
42            for tier in (2160, 1440, 1080, 720, 480, 360, 240, 144):
43                if height >= tier * 0.95:
44                    return f"{tier}p"
45            return f"{height}p"
46
47
48        def build_ytdlp_command(url: str, dest_path: str, max_height: Optional[int] = None,
49                                cookies_from_browser: Optional[str] = None,
50                                cookies_file: Optional[str] = None,
51                                remote_components: Optional[str] = "ejs:github",
52                                player_client: Optional[str] = "default") -> List[str]:
53            """Build the yt-dlp argv for a best-quality download.
54
55            `max_height`, when set, caps resolution (e.g. avoid pulling 4K); left unset
56            it fetches the absolute best available. Pure/argv-only so it is unit-testable

```

```

57     without invoking the network.
58
59     `player_client` selects YouTube's player client (default `default` = yt-dlp's
60     maintained client set, which serves real downloadable 1080p DASH with auth
61     cookies). Avoid `web`/`web_safari`: YouTube forces those into SABR streaming,
62     which strips download URLs and silently drops to the lowest stream (the 144p
63     bug). Harmless for non-YouTube URLs (the extractor-arg is ignored). Set to
64     None to omit the arg entirely.
65
66     `remote_components` enables yt-dlp's EJS challenge solver (default
67     `ejs:github`), the JS-runtime (deno) path for solving YouTube's n-challenge.
68     Set to None to disable (e.g. for non-YouTube sources or offline tests).
69     """
70     fmt = _FORMAT
71     if max_height:
72         # Cap each stream selector; keep the pre-muxed fallback capped too.
73         fmt = f"bv*[height<={max_height}]+ba/b[height<={max_height}]/b"
74     cmd = [
75         "yt-dlp",
76         "-f", fmt,
77         "-S", _SORT,
78         "--merge-output-format", "mp4",
79         "--no-playlist",
80         "--no-progress",
81         "-o", dest_path,
82     ]
83     if player_client:
84         cmd += ["--extractor-args", f"youtube:player_client={player_client}"]
85     if remote_components:
86         cmd += ["--remote-components", remote_components]
87     if cookies_from_browser:
88         cmd += ["--cookies-from-browser", cookies_from_browser]
89     if cookies_file:
90         cmd += ["--cookies", cookies_file]
91     cmd.append(url)
92     return cmd
93
94
95     def _env_with_deno_on_path() -> dict:
96         """Return an environment with the default deno install dir on PATH.
97
98         The EJS challenge solver needs a JS runtime (deno). The official installer
99         drops it at ~/.deno/bin, but a shell started before install won't have it on
100        PATH yet. We add it here so `fetch` works without requiring a terminal
101        restart ? without clobbering a deno the user already has elsewhere.
102        """
103        env = os.environ.copy()
104        if shutil.which("deno"):
105            return env # already discoverable
106        deno_bin = os.path.join(os.path.expanduser("~"), ".deno", "bin")
107        if os.path.isdir(deno_bin):
108            env["PATH"] = deno_bin + os.pathsep + env.get("PATH", "")
109        return env
110
111
112     def probe_resolution_fps(path: str) -> Tuple[Optional[int], Optional[int],
Optional[float]]:
113         """Return (width, height, fps) of a video file via ffprobe, best-effort.
114
115         Returns (None, None, None) if ffprobe is unavailable or the file can't be
116         parsed, so a missing ffprobe degrades gracefully rather than crashing a fetch.
117         """
118         if not shutil.which("ffprobe") or not os.path.exists(path):
119             return (None, None, None)
120         try:

```

```

121         out = subprocess.run(
122             ["ffprobe", "-v", "error", "-select_streams", "v:0",
123              "-show_entries", "stream=width,height,avg_frame_rate",
124              "-of", "default=noprint_wrappers=1:nokey=1", path],
125             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
126             ).stdout.split()
127     except (subprocess.SubprocessError, OSError):
128         return (None, None, None)
129
130     width = int(out[0]) if len(out) > 0 and out[0].isdigit() else None
131     height = int(out[1]) if len(out) > 1 and out[1].isdigit() else None
132     fps = None
133     if len(out) > 2 and "/" in out[2]:
134         num, den = out[2].split("/")
135         try:
136             den_f = float(den)
137             fps = round(float(num) / den_f, 3) if den_f else None
138         except ValueError:
139             fps = None
140     return (width, height, fps)
141
142
143 @dataclass
144 class DownloadResult:
145     success: bool
146     path: Optional[str] = None
147     size_mb: float = 0.0
148     width: Optional[int] = None
149     height: Optional[int] = None
150     fps: Optional[float] = None
151     resolution_label: Optional[str] = None
152     error: Optional[str] = None
153
154
155 def download_best_video(url: str, dest_path: str, max_height: Optional[int] = None,
156                        max_size_mb: Optional[float] = None,
157                        cookies_from_browser: Optional[str] = None,
158                        cookies_file: Optional[str] = None,
159                        player_client: Optional[str] = "default") -> DownloadResult:
160     """Download the best-resolution video for `url` to `dest_path` (MP4).
161
162     On success, ffprobes the file and reports actual width/height/fps. If
163     `max_size_mb` is exceeded the file is removed and a failed result returned,
164     so callers never register an over-budget download.
165     """
166     os.makedirs(os.path.dirname(os.path.abspath(dest_path)), exist_ok=True)
167     cmd = build_ytdlp_command(url, dest_path, max_height=max_height,
168                              cookies_from_browser=cookies_from_browser,
169                              cookies_file=cookies_file,
170                              player_client=player_client)
171     try:
172         subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
173                        text=True, check=True, env=_env_with_deno_on_path())
174     except FileNotFoundError:
175         return DownloadResult(success=False, error="yt-dlp not found on PATH")
176     except subprocess.CalledProcessError as e:
177         return DownloadResult(success=False, error=(e.stderr or str(e)).strip()[:500])
178
179     if not os.path.exists(dest_path):
180         return DownloadResult(success=False, error="yt-dlp reported success but no file
was produced")
181
182     size_mb = os.path.getsize(dest_path) / (1024 * 1024)
183     if max_size_mb is not None and size_mb > max_size_mb:
184         os.remove(dest_path)

```

```

185         return DownloadResult(
186             success=False, size_mb=size_mb,
187             error=f"downloaded size {size_mb:.1f} MB exceeds limit {max_size_mb:.1f}
MB",
188         )
189
190     width, height, fps = probe_resolution_fps(dest_path)
191     return DownloadResult(
192         success=True, path=dest_path, size_mb=size_mb,
193         width=width, height=height, fps=fps,
194         resolution_label=height_to_label(height),
195     )
196

```

13. assistant (2026-06-07T06:52:06.900Z)

Let me check the ADR-005 license constraint, the indexer's dependency manifest (to confirm Jinja2/pydantic), the collector's dependencies, and the AI provider timing return shape and WASB manifest to confirm harvest sources.

14. user (2026-06-07T06:52:10.336Z)

```

Exit code 1
AUDIO_E1_FINDINGS.md
AUDIO_RALLY_DETECTION_PLAN.md
BACKLOG.md
CODE_MAP.md
DECISIONS.md
EVALUATION.md
GOLDEN_SET_IMPLEMENTATION.md
GOLDEN_SET_PHASE1_VIDEOS.md
GOLDEN_SET_VENDORING.md
HOW_RALLY_DETECTION_WORKS.md
MONETIZATION_AUDIT.md
RALLY_DETECTION_RESEARCH.md
ROADMAP.md
TRACKNET_WSL_SETUP.md
VIDEO_RECORDING_GUIDELINES.md
---REQ-INDEXER---
/usr/bin/bash: line 1: $null: ambiguous redirect
---PYPROJECT-INDEXER---
/usr/bin/bash: line 1: $null: ambiguous redirect
---REQ-COLLECTOR---
/usr/bin/bash: line 1: $null: ambiguous redirect
---PYPROJECT-COLLECTOR---
/usr/bin/bash: line 1: $null: ambiguous redirect

```

15. assistant (2026-06-07T06:52:12.995Z)

That was the Bash tool (POSIX). Let me use the right tooling for each lookup.

16. user (2026-06-07T06:52:14.030Z)

```

requirements.txt
setup.py

```

17. user (2026-06-07T06:52:14.954Z)

```

C:\Users\avidu\Projects\sports-data-collector\pyproject.toml

```

18. user (2026-06-07T06:52:15.862Z)

```

232:## ADR-005: Permissive Dependency Stack for the Hosted Monetization Path
233-

```

234-**Date:** 2026-05-30
235-**Status:** Accepted
236-
237-### Context
238-
239-[Omitted long context line]
240-
241-### Decision
242-
243-[Omitted long context line]
244-2. **Shuttle tracking stays permissive.** **WASB-SBDT and TrackNetV3 are MIT (CLEAR for commercial use)** ? this resolves the licensing unknown flagged in ADR-001/ADR-004. **MonoTrack (Adobe Research, noncommercial) and YOLO-NAS weights (Deci, noncommercial) are banned.** Train our own checkpoints (or confirm per-checkpoint terms), since distributed research weights/datasets aren't covered by the MIT *code* grants.
245-3. **LLM step ? keep all three strategies open:** (A) eliminate via an offline temporal segmenter (**ActionFormer, MIT** ? the Phase-4 direction), (B) self-host **Qwen2.5-VL-7B/32B (Apache-2.0)** or **InternVL2.5 (MIT)**, or (C) keep **Gemini 2.5 Flash on the paid tier** (required ? the free tier trains on and human-reviews uploaded video). The `google-genai` SDK itself is Apache-2.0 (clear).
246-4. **Emit royalty-free output.** Standardize highlight output on **AV1 (+ Opus)** to avoid AVC/HEVC encode-side patent exposure; provision an **Ada-class GPU (L4/L40)** for hardware AV1 (NVENC), falling back to **SVT-AV1 (CPU, BSD-3-Clause-Clear)** on T4/A10/A100.
247-
248-### Consequences
249-
250-- Eliminates the only two hard legal blockers; the rest of the stack (torch, opencv, fastapi, numpy, lapx, ffmpeg-as-subprocess, google-genai SDK) is already permissive.
251-- Detector swap is contained to `backend/indexer/yolo_hybrid.py` (the `from ultralytics import YOLO` load + the `model.track(...)` call) plus `config.json` knobs and `tests/test_yolo_hybrid.py` mocks. Stage-1 feature outputs (velocity/active-frame/track-id stats) and Stages 2?4 are detector-agnostic and unchanged.
252-- We now own the frame loop + tracking (ultralytics did both internally). Use a permissive ByteTrack (e.g. `supervision`, MIT, or the MIT ByteTrack repo) on top of `lapx` (already a BSD dep).
253-- Residual items needing legal sign-off: exact codec-pool royalty thresholds (AVC/HEVC decode of uploads), Ultralytics Enterprise pricing, and per-checkpoint weight licenses ? tracked in `docs/MONETIZATION_AUDIT.md` §7.
254-- The `yolo_hybrid` registry key is now a slight misnomer (no YOLO); keep it for back-compat or rename in a follow-up.
255-
256-
257-
--
265:[Omitted long matching line]
266-
267-Decisions locked with the owner: **licensed/owned video only** (no user uploads to vendors) . **rally** `[start,end]` annotation depth . **tiered turnaround** (fast automated + slow human) . **vendor bake-off** to choose . **Bangalore/India-local preference**.
268-
269-### Decision
270-
271-1. **One scorer, three jobs.** A vendor's `[start,end]` output is structurally identical to a prediction, so `metrics.py` grades product-vs-truth, vendor-vs-key, and annotator-vs-annotator (IAA) with no new scoring code.
272-2. **Select vendors by measured quality-per-dollar**, via a bake-off: write a precise rally-definition guideline, build a small internal gold key (5?10 licensed clips), send the same clips to 2?3 vendors (favoring Bangalore/India), and grade every submission with our own harness. The numbers are the decision.
273-3. **Pluggable** `AnnotationProvider` registry mirroring the existing registry pattern: `FileProvider` (today's manual/ShuttleSet flow), `HumanVendorProvider` (slow tier ? authoritative truth), `AutomatedProvider` (fast tier ? independent model oracle).
274-4. **Regression = existing harness + one new component.** `regress(video)` calls a provider for ground truth, runs the production pipeline, and scores with the unchanged `metrics.py` against a per-video `baseline.json`.

275-5. ****Privacy by construction.**** Golden sets use only licensed/owned footage; all third-party video egress flows through the single provider boundary where DPA / no-reuse / delete-on-delivery / access-logging controls live.

276-

277-### Consequences

278-

279-- Eliminates the "human views user video" concern for golden-set/regression work, because user uploads never enter the pipeline.

280-- ****The oracle problem is a first-class caveat:**** the fast-tier `AutomatedProvider` must be a **different model lineage** than production (else shared blind spots make the test falsely green); fast-tier alarms escalate to the slow human tier before blocking a release.

281-- New docs: `docs/GOLDEN\_SET\_VENDORING.md` (the full process v1) and `docs/VIDEO\_RECORDING\_GUIDELINES.md` (a tracked draft: capture-quality thresholds to be derived empirically from precision/recall, since input quality bounds output quality ? cf. ADR-004's broadcast-camera finding).

282-- The vendor shortlist itself is ****pending a primary-source research pass**** (favoring India/Bangalore) and is intentionally left blank until verified ? no un-sourced vendor claims.

283-- ****Honest limitation:**** a licensed/owned corpus may not fully represent real user capture, so regression greens are strong evidence, not guarantees ? the recording-guidelines work aims to narrow that gap.

284-

285-### Update (2026-06-02): shortlist researched; Roora selected for the initial pilot

286-

287-- The ****vendor shortlist**** is filled in (GOLDEN_SET_VENDORING.md §6, primary-source pass 2026-05-30).

288-- The founder selected ****Roora ML Pvt Ltd**** (HSR Layout, Bengaluru) for the ****initial paid pilot****, diverging from the doc's iMerit/Taskmonk default toward a Bangalore-local, startup-friendly shop. Roora's existence, image/video/temporal capability, and location are verified (rooragr.com); ****pilot quality is still to be measured**** ? run as a ****single-vendor pilot first**** (the pilot **is** the calibration), not a multi-vendor bake-off.

289-- ****Step 0 ? the annotation guideline ? is authored**** (the highest-leverage artifact): collector repo `docs/annotation/` ? `ROORA\_BRIEF.md` (rally CSV schema), `ANNOTATION\_GUIDE.md` (illustrated guide), plus intake-manifest and NDA templates.

290-- ****Pilot scope is 3 sports**** (badminton, pickleball, table tennis); the collector's `import-local` ingestion for all three is implemented (collector ADR-001).

--

327:6. Stay permissive (ADR-005): classical DSP (numpy/scipy, BSD), no restrictive weights; any later model trained

328- in-house.

329-

330-### Consequences

331-

332-- New `backend/audio/` module (extraction + onset/peak + confidence), feeding `distill\_local` and scored by the

333- existing `metrics.py` / golden loaders. Adds a light ****scipy**** dependency.

334-- ****All audio measurement is blocked on golden eval labels**** ? which the VLC rally-annotator (now a standalone

335- MIT, multi-sport open-source repo: <https://github.com/avidullu/rally-annotator>) lets the owner self-produce,

336- and the in-repo `--labels` loaders / collector `import-local` ingest.

337-- Sport-generic: tennis/table-tennis hits are **louder**, so the cue should transfer at least as well there.

338-- ****Honest risk:**** if E1 shows shuttle SNR is too poor on real amateur audio, audio is deferred ? this ADR

339- commits to the **experiment and direction**, not to a guaranteed win.

340-

341-### Integration architecture: modular fusion now (Option A), an owned end-to-end model later (Option B)

342-

343-A recurring design question: do we ****feed audio into WASB**** so it refines its own output, or ****keep audio as a**

344-separate signal fused in our code**? The answer today is the latter ? and understanding **why** sets up the

345-long-term route.

346-

347-****Why fusion lives in our rally layer, not inside WASB (Option A ? now).**** WASB is a
****frozen, vision-only**
348-shuttle ***detector**** (image frames ? per-frame shuttle `(x, y, visibility)`; MIT weights we
run, not train).
349-It has no audio pathway and it does not decide rallies at all ? ****rally detection is our own**
layer** on top of
350-its trajectory (`trajectory\_to\_action\_windows` ? gate ? the distilled classifier). Making
WASB consume audio
351-would mean bolting on an audio encoder and ****retraining the network****, which needs per-frame
352-shuttle-coordinate-***plus***-audio labels we don't have (we only have rally `[start,end]`
labels) and would fork

19. assistant (2026-06-07T06:52:20.188Z)

I now have a solid grasp of the licensing posture (ADR-005 strongly permissive, MIT/BSD/Apache only). Let me confirm the indexer's requirements.txt contents (to verify Jinja2 + pydantic deps), the collector's pyproject deps, and the README FAQ structure plus the AI provider timing return shape.

20. user (2026-06-07T06:52:20.668Z)

```
1      fastapi>=0.111.0
2      uvicorn>=0.30.1
3      opencv-python>=4.9.0.80
4      numpy>=1.26.4
5      pydantic>=2.7.1
6      jinja2>=3.1.4
7      python-dotenv>=1.0.0
8      google-generativeai>=2.4.0
9      torch>=2.12.0
10     torchvision>=0.27.0
11     supervision>=0.21.0,<0.30.0
12
```

21. user (2026-06-07T06:52:21.175Z)

```
1      [build-system]
2      requires = ["setuptools>=61.0.0"]
3      build-backend = "setuptools.build_meta"
4
5      [project]
6      name = "sports-data-collector"
7      version = "0.1.0"
8      description = "A collector and curator for sports video datasets with sport-specific
schemas and license checks."
9      readme = "README.md"
10     requires-python = ">=3.9"
11     dependencies = [
12         "pydantic>=2.0.0",
13         "pyyaml>=6.0",
14         "tabulate>=0.9.0"
15     ]
16
17     [project.optional-dependencies]
18     # Video acquisition (`fetch` / crawler). Also requires the external binaries
19     # ffprobe (FFmpeg) and, for YouTube high-res, a JS runtime such as deno ?
20     # neither is pip-installable; see README "External requirements".
21     fetch = [
22         "yt-dlp>=2024.1.0"
23     ]
24     dev = [
25         "pytest>=7.0.0",
26         "pytest-cov>=4.0.0"
27     ]
```

22. user (2026-06-07T06:52:22.314Z)

166:## ? Frequently Asked Questions (FAQ)
 168:### Q1: The server complains that `ffprobe` is not found.
 171:### Q2: Why does my video fail the Relevance check?
 174:### Q3: How do I add a new custom validator to the pipeline?
 180:### Q4: Why does the CLI or server output freeze or take a long time on large videos?
 187:### Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?
 190:### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB video file?
 197:### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
 219:### Q8: How does the `yolo\_hybrid` pipeline actually save money?
 229:### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?

23. assistant (2026-06-07T06:52:26.756Z)

Both repos already have pydantic; indexer has jinja2; collector has pyyaml. The FAQ Q1-Q9 map cleanly to footgun flags. Now let me confirm the WASB manifest harvest source, the AI provider timing return, and the compiler skip-tuple shape, plus the collector DB Video model fields and the import-local/fetch CLI handlers.

24. user (2026-06-07T06:52:28.298Z)

```

1      """WASB-based hybrid play segmenter.
2
3      Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4      (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5      (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6      badminton weights and runs today, so this is the live shuttle-motion substrate.
7      Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9      For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10     windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11     are what get tuned against golden rally labels.
12     """
13     import os
14     import time
15     import logging
16     import concurrent.futures
17     from typing import List, Dict, Any
18
19     import cv2
20
21     from backend.indexer.base import BasePlaySegmenter, segmenter_registry
22     from backend.utils.video import get_video_duration, extract_video_chunk
23     from backend.sports import sport_registry
24     from backend.providers import provider_registry
25     from backend.detectors.wasb_runner import WasbRunner, WasbConfig
26
27     logger = logging.getLogger(__name__)
28
29
30     def _fmt_ts(seconds: float) -> str:
31         seconds = max(0, int(seconds))
32         h, rem = divmod(seconds, 3600)
33         m, s = divmod(rem, 60)
34         return f"{h:02d}:{m:02d}:{s:02d}"
35
36
37     class WasbHybridSegmenter(BasePlaySegmenter):
38         @property

```

```

39         def name(self) -> str:
40             return "wasb_hybrid"
41
42         @property
43         def description(self) -> str:
44             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
45                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
46
47         @staticmethod
48         def _probe_fps_width(video_path: str) -> tuple:
49             cap = cv2.VideoCapture(video_path)
50             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
51             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
52             cap.release()
53             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
54
55         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
56                           max_frames: int = None) -> List[Dict[str, Any]]:
57             sport_name = self.config.get("sport", "badminton")
58             try:
59                 sport_profile = sport_registry.get(sport_name)
60             except KeyError as e:

```

25. user (2026-06-07T06:52:29.961Z)

```

167:         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,

```

26. assistant (2026-06-07T06:52:38.360Z)

The provider timing metrics (`upload\_time/process\_time/gen\_time`) are discarded at the segmenter level (the tuple's second element is `\_`). Let me confirm the WASB manifest path and the candidate-persistence call, plus check the collector's Video DB model and the import-local/fetch handlers.

27. user (2026-06-07T06:52:39.672Z)

```

60             except KeyError as e:
61                 logger.error(str(e))
62                 return []
63
64             idx_cfg = self.config.get("indexing", {})
65             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
66             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
67
68             api_key_env_var = f"{provider_name.upper()}_API_KEY"
69             api_key = os.environ.get(api_key_env_var)
70             if not api_key:
71                 logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
72                 return []
73             try:
74                 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
75             except KeyError as e:
76                 logger.error(str(e))
77                 return []
78
79             video_duration = get_video_duration(video_path)
80             if video_duration <= 0:

```

```

81         logger.error("Invalid video duration.")
82         return []
83     fps, frame_width = self._probe_fps_width(video_path)
84
85     wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
86     runner = WasbRunner(wasb_cfg)
87
88     w = idx_cfg.get("wasb", {})
89     velocity_thresh = w.get("velocity_threshold", 0.02)
90     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
91     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
92     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
93     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
94     log_candidates = idx_cfg.get("log_yolo_candidates", True)
95     store_candidates = idx_cfg.get("store_yolo_candidates", True)
96
97     detection_params = {
98         "detector": "wasb_hybrid",
99         "velocity_thresh": velocity_thresh,
100        "merge_gap": merge_gap,
101        "min_action_window_duration": min_window_duration,
102        "weights_path": wasb_cfg.weights_path,
103    }
104
105    ok, msg = runner.healthcheck()
106    if not ok:
107        logger.error("WASB WSL environment not ready: %s", msg)
108        return []
109    logger.info("WASB WSL env OK: %s", msg)
110
111    # --- Phase 2: shuttle trajectory -> candidate rally windows ---
112    t_start = time.time()
113    out_dir = os.path.join("output", "wasb")
114    csv_path = runner.run_predict(video_path, out_dir)
115    if not csv_path:
116        logger.error("WASB produced no trajectory; aborting.")
117        return []
118
119    points = runner.parse_trajectory_csv(csv_path)
120    logger.info("Parsed %d trajectory points (%d visible).",
121               len(points), sum(1 for p in points if p.visible))
122
123    all_candidate_windows = runner.trajectory_to_action_windows(
124        points, fps=fps, frame_width=frame_width, chunk_start=0.0,
125        velocity_thresh=velocity_thresh, merge_gap=merge_gap,
126        min_window_duration=min_window_duration, chunk_index=0,
127    )
128    logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
129               time.time() - t_start, len(all_candidate_windows))
130
131    if log_candidates:
132        for cw in all_candidate_windows:
133            logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
134                       f"({cw['end'] - cw['start']:.1f}s) | "
135                       f"peak_v={cw['peak_velocity']:.3f} "
136                       f"mean_v={cw['mean_velocity']:.3f}")
137
138    candidate_ids = [None] * len(all_candidate_windows)
139    if store_candidates:
140        for i, cw in enumerate(all_candidate_windows):
141            stats = {
142                "peak_velocity": cw["peak_velocity"], "mean_velocity":

```

```

142 cw["track_id_count"],
143     }
144     candidate_ids[i] = self.db.add_candidate_segment(
145         video_id, detector="wasb_hybrid", start_time=cw["start"],
146         end_time=cw["end"],
147         chunk_index=cw["chunk_index"], confidence=min(1.0,
148         cw["peak_velocity"]),
149         stats=stats, detection_params=detection_params,
150     )
151     if not all_candidate_windows:
152         return []
153     # --- Phase 3: AI provider handoff (shared pattern) ---
154     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
155     os.makedirs(handoff_dir, exist_ok=True)
156     logger.info("Phase 3: %s validation on %d candidate windows...",
157         provider_name, len(all_candidate_windows))
158
159     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
160         w_start = max(0, window["start"] - handoff_padding)
161         w_end = min(video_duration, window["end"] + handoff_padding)
162         w_dur = w_end - w_start
163         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
164         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
165             reencode=True):
166             return []
167         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
169             label=f"Handoff {wi+1}")
170         valid = []
171         for seg in segments:
172             try:
173                 seg["start_time"] = float(seg["start_time"]) + w_start
174                 seg["end_time"] = float(seg["end_time"]) + w_start
175                 seg["_candidate_id"] = candidate_ids[wi]
176                 valid.append(seg)
177             except (ValueError, TypeError, KeyError) as e:
178                 logger.warning("Dropping segment with unparseable timestamps: %s ?
179 %s", seg, e)
180         try:
181             os.remove(w_path)
182         except OSError:
183             pass
184         return valid
185
186     ai_segments: List[dict] = []
187     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
188     executor:
189         futures = [executor.submit(process_candidate_window, wi, cw)
190             for wi, cw in enumerate(all_candidate_windows)]
191         for future in futures:
192             ai_segments.extend(future.result())

```

28. assistant (2026-06-07T06:52:43.199Z)

This confirms the key plumbing fact: AI timing metrics (`segments`, `\_`) are discarded inside the segmenter, candidates are persisted to DB (harvestable via `query\_candidate\_segments`), and the WASB manifest lives at `output/wasb/manifest.json`. Now let me check the collector's Video DB model fields and the import-local/fetch CLI handlers to complete the collector integration picture.

29. user (2026-06-07T06:52:45.762Z)

C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:5: video_id: str

```

C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:29:class
VideoMetadata(BaseModel):
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:30:    video_id: str =
Field(..., description="Unique alphanumeric identifier for the video")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:32:    video_url:
Optional[str] = Field(None, description="Remote URL of the video (if available)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:33:    local_path:
Optional[str] = Field(None, description="Local path to video file (if downloaded or custom)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:34:    license_type:
LicenseType
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:36:    is_commercial:
bool = Field(False, description="Flag indicating if the license allows commercial use")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:37:    status:
CurationStatus = CurationStatus.STAGING
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:38:    match_name: str =
Field(..., description="Name of the match/event")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:39:    fps:
Optional[float] = Field(None, description="Frames per second")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:40:    resolution:
Optional[str] = Field(None, description="Resolution string (e.g. 1080p, 720p)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:5:    video_id: str
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:5:    video_id: str
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:5:    video_id:
str
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:5:    video_id: str

```

30. user (2026-06-07T06:52:47.055Z)

```

52:    def import_local(self, args):
330:    def fetch(self, args):
810:    subparsers.add_parser("status", help="Display summary status of sports metadata
database")
813:    subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
816:    import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
827:    fetch_parser = subparsers.add_parser(
858:    cvat_parser = subparsers.add_parser(
875:    vd_parser = subparsers.add_parser(
888:    export_parser = subparsers.add_parser(

```

31. assistant (2026-06-07T06:52:50.879Z)

I have enough to harden the design. Let me read the `import\_local` handler and the CLI dispatch/main to nail the exact integration seam, plus confirm the collector has no Jinja2 (will need it as a new dep there).

32. user (2026-06-07T06:52:51.807Z)

```

52    def import_local(self, args):
53        """Manually registers a local video and annotation file into the database."""
54        sport_str = args.sport.lower()
55        if sport_str not in self.config.get("supported_sports", ["badminton"]):
56            print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57            sys.exit(1)
58
59        # Validate video path
60        if not os.path.exists(args.video_path):
61            print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63        # Validate annotation file
64        if not os.path.exists(args.annotations_path):

```

```

65         print(f"Error: Annotations file '{args.annotations_path}' not found.")
66         sys.exit(1)
67
68     match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69     video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71     # Parse license
72     try:
73         license_type = LicenseType(args.license)
74     except ValueError:
75         print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76         license_type = LicenseType.UNKNOWN
77
78     is_commercial = self.validator.check_license_commercial(license_type)
79
80     # 1. Parse Annotations
81     annotations = []
82     try:
83         annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84     except Exception as e:
85         print(f"Error parsing annotations: {e}")
86         sys.exit(1)
87
88     # 2. Run Validation
89     is_valid = True
90     errors = []
91     if sport_str == "badminton":
92         is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93     elif sport_str == "tennis":
94         is_valid, errors = self.validator.validate_tennis_points(annotations)
95     elif sport_str == "cricket":
96         is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97     elif sport_str == "pickleball":
98         is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99     elif sport_str == "tabletennis":
100         is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102     if not is_valid:
103         print(f"Error: Annotations failed validation!")
104         for err in errors:
105             print(f" - {err}")
106         sys.exit(1)
107
108     # 3. Create Video Metadata
109     video_meta = VideoMetadata(
110         video_id=video_id,
111         sport=SportType(sport_str),
112         video_url=None,
113         local_path=args.video_path,
114         license_type=license_type,
115         license_url=None,
116         is_commercial=is_commercial,
117         status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118         match_name=match_name,
119         fps=args.fps,
120         resolution=args.resolution
121     )
122
123     # 4. Insert into Database
124     self.db.insert_video(video_meta)
125
126     count = 0

```

```

127         if sport_str == "badminton":
128             count = self.db.insert_badminton_rallies(annotations)
129         elif sport_str == "tennis":
130             count = self.db.insert_tennis_points(annotations)
131         elif sport_str == "cricket":
132             count = self.db.insert_cricket_deliveries(annotations)
133         elif sport_str == "pickleball":
134             count = self.db.insert_pickleball_rallies(annotations)
135         elif sport_str == "tabletennis":
136             count = self.db.insert_tabletennis_rallies(annotations)
137
138         print(f"\nSuccessfully imported local video '{match_name}':")
139         print(f"  - Video ID: {video_id}")
140         print(f"  - Sport: {sport_str}")
141         print(f"  - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142         print(f"  - Registered {count} annotated segments.\n")
143
144     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
145         """Parses simple JSON or CSV annotations files."""
146         parsed_items = []
147         ext = os.path.splitext(filepath)[1].lower()
148
149         if ext == '.json':
150             with open(filepath, 'r', encoding='utf-8') as f:
151                 data = json.load(f)
152
153             if not isinstance(data, list):
154                 raise ValueError("JSON file must contain a list of objects.")
155
156             # Uses the same safe casts as the CSV path: a missing/blank required
157             # time raises a clear ValueError (caught by the caller) instead of a
158             # bare KeyError, and optional ints degrade to sensible defaults.
159             for idx, obj in enumerate(data):
160                 if sport == "badminton":
161                     parsed_items.append(BadmintonRally(
162                         video_id=video_id,
163                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
164                         start_time=_safe_float(obj.get("start_time")),
165                         end_time=_safe_float(obj.get("end_time")),
166                         score_before=obj.get("score_before"),
167                         score_after=obj.get("score_after"),
168                         shots_count=_safe_int(obj.get("shots_count"), None),
169                         ending_reason=obj.get("ending_reason"),
170                         last_shot_outcome=obj.get("last_shot_outcome")
171                     ))

```

33. assistant (2026-06-07T06:52:55.723Z)

I now have all the integration seams confirmed across both repos. I have enough to write a concrete, hardened plan. Let me write it to the plan file.

34. assistant (2026-06-07T06:55:48.584Z)

Rich Per-Video Observability Report ? Implementation Plan

Spans 3 repos: NEW shared library (`obsreport`), `badminton-highlight-indexer` (Repo A), `sports-data-collector` (Repo B).

Goal: after every video is processed/ingested, ALWAYS emit (1) a rich technical report capturing every stage + details, and (2) a customer-friendly report (metadata + meta-level highlights only). Both report types from both tools. Written next to the video by convention, with a configurable override. Schema unified via a new private GitHub repo providing a reusable "report

config language."

0. Verified facts (from code read, not assumed)

These ground the design; cite when adapting.

- **Indexer deps already present** (``requirements.txt``): ``pydantic>=2.7.1``, ``jinja2>=3.1.4``. So Jinja2 + pydantic are free in Repo A.
- **Collector deps** (``pyproject.toml``): ``pydantic>=2.0.0``, ``pyyaml>=6.0``, ``tabulate``. No Jinja2 ? adding the shared lib brings Jinja2 transitively (acceptable; BSD).
- **``registry.run_all``** (``backend/validators/base.py:42-56``) builds a local ``context={}``, populates ``context['ffprobe_meta']`` via `FormatValidator`, returns ONLY ``List[ValidationResult]``. The context (with probe metadata) is discarded. Confirmed: to get probe metadata into a report, either surface ``context`` or re-probe.
- **AI provider timing discarded at segmenter level**: ``wasb_hybrid.py:167`` does ``segments, _ = provider.analyze_video(...)`` ? the ``{upload_time,process_time,gen_time}`` metrics dict is dropped. Same pattern in tracknet/gemini hybrids.
- **Candidates ARE persisted**: ``wasb_hybrid.py:144`` ``db.add_candidate_segment(...)``; queryable via ``db.query_candidate_segments(detector=)``. WASB manifest at ``output/wasb/manifest.json`` (relative cwd path).
- **CLI path** ``backend/main.py:312-352``; **API path** ``process_video @POST /api/process`` ``backend/main.py:88-134``. Both: ``get_file_md5`` ? ``registry.run_all`` ? ``db.add_video`` ? ``segmenter_registry.get(name)`` ? ``segmenter.process_video``. Note ``add_video`` is ``INSERT OR REPLACE`` + FK CASCADE.
- **Collector existing report module** ``src/parsers/cvat/rally_cvat.py`` is the reuse target: ``build_summary()`` (JSON dict), ``render_validation_md()`` (MD), ``write_review_csv()``, ``detect_issues()`` (severity model: blocker/high/medium/low; ``Issue{severity,code,message,rally_number}``), ``ConversionResult{records,issues,fps,step,meta}``. ``validate-delivery`` (``curate.py:562-617``) already writes ``<id>_report.json/.md`` + ``<id>_review.csv``. ``export`` (``curate.py:428-524``) writes per-video CSV + ``manifest.json``.
- **Collector Video model** ``src/core/schemas/base.py:29-40`` `VideoMetadata: `video_id, video_url, local_path, license_type, is_commercial, status, match_name, fps, resolution``. ``probe_resolution_fps`` in ``src/core/downloader.py:112``. ``DownloadResult`` (``downloader.py:143``) carries ``success, size_mb, width, height, fps, resolution_label, error``.
- **License posture (ADR-005)**: hard permissive-only. The stack pydantic(MIT)/Jinja2(BSD)/PyYAML(MIT)/jsonschema(MIT) is clean. Soda=ELV2 disqualified. GE/Evidently/ydata/Frictionless = Apache/MIT but tabular/dataframe-shaped ? wrong fit confirmed.
- **README FAQ Q1-Q9** (``README.md:166+``) maps cleanly to footgun flags: Q1 ffprobe missing, Q2 relevance fail, Q5 PTS/keyframe, Q6/Q9 debug_duration truncation, Q7 AI provider config (API key), Q8 yolo_hybrid economics. These are the citation targets for flag messages.

1. Library vs framework decision ? VALIDATED, with refinements

Verdict: build the small in-house library. Confirmed correct. **Rationale:**

1. **Shape mismatch is decisive, not just license.** GE/Evidently/ydata/Frictionless model ``tabular datasets / data packages`` (columns, distributions, expectations over rows). The domain object here is a **per-video pipeline run = an ordered list of heterogeneous stages**, each with a free-form metrics blob and a verdict. Forcing that into a dataframe expectation suite is an impedance mismatch that would itself become a house of cards. The "config language" you actually want is "describe sections/fields/audience + declarative footgun?flag rules," which none of those tools express naturally.
2. **License is a hard gate** (ADR-005) and independently disqualifies Soda. The permissive primitives (pydantic/Jinja2/PyYAML/jsonschema) are already in-tree, so zero new license risk and near-zero new transitive surface.
3. **House-of-cards containment**: the in-house lib is ~6-8 small modules with a frozen typed envelope and pure renderers. It has no I/O coupling to either pipeline except a thin recorder API. That is **far** less fragile than wrapping a heavy framework whose upgrade cadence and API churn you don't control.

```

**Refinements / cautions to bake in:**
- **Do NOT vendor `json-schema-for-humans` or `yaml-to-markdown` into the core dependency closure.** Keep the core renderer set to hand-written Jinja2 templates (you control wording precisely, which the FAQ-debuggability eval *requires*). Treat those MIT renderers as optional dev-time helpers for generating the schema doc only, not runtime deps.
- **`jsonschema` is optional**: pydantic v2 already does validation and can emit JSON Schema (model_json_schema()). Use pydantic as the single source of truth for the envelope; only pull jsonschema if you need to validate *externally authored config files* against a published schema. Recommendation: defer jsonschema to Phase 2+, validate config with pydantic models first.
- **The "config language" must stay declarative-but-tiny.** Resist building a general expression engine. Use a *restricted, safe* rule grammar (see §2.4) ? NOT eval(), NOT a DSL parser. A list of typed comparison clauses is enough for every footgun listed.

```

2. The shared library ? design

2.1 Repo name

Recommend `**sports-obsreport**` (clear domain tie) or generic `**obsreport**` if reuse beyond sports is intended. Plan uses package name `**obsreport**`, repo `sports-obsreport`. Private GitHub repo; MIT (own code).

Layout:

```

...
sports-obsreport/
  pyproject.toml          # name=obsreport; deps: pydantic>=2, jinja2>=3.1, pyyaml>=6
  src/obsreport/
    __init__.py           # public API: ReportEnvelope, RunRecorder, render_*, load_spec
    envelope.py           # the typed pydantic models (the schema)
    spec.py               # the declarative config-language loader + models
    rules.py              # the restricted rule engine (footgun -> flag)
    recorder.py           # RunRecorder (in-flight) + harvest helpers (post-hoc)
    render/
      json.py              # canonical JSON (source of truth)
      markdown_technical.py # Jinja2 -> MD technical
      markdown_customer.py  # Jinja2 -> MD/HTML customer
      templates/*.j2
    io.py                  # path resolution (next-to-video + override) + safe-write
    version.py             # SCHEMA_VERSION const
  specs/                  # shipped default specs (YAML)
    indexer.yaml
    collector.yaml
  tests/                  # pytest, fully offline
  ...

```

2.2 Typed report ENVELOPE (`envelope.py`, pydantic v2)

Single source of truth. JSON is serialized from this; both Markdown renderers consume this.

...

```
SCHEMA_VERSION = "1.0"
```

```

class Identity(BaseModel):
    schema_version: str
    tool: str          # "badminton-highlight-indexer" | "sports-data-collector"
    tool_version: str | None
    report_kind: str    # "run"
    video_id: str       # MD5 (indexer) or human id (collector) ? document which
    video_id_kind: str  # "file_md5" | "human_id"
    video_path: str | None
    video_url: str | None # collector remote case
    generated_at: str    # ISO8601 UTC
    run_id: str          # uuid4 per run (idempotency / cross-run diff key)

```

```

class Metric(BaseModel):          # one measured value, audience-tagged
    key: str
    value: Any
    unit: str | None = None
    audience: Literal["technical","customer","both"] = "technical"

class Stage(BaseModel):
    name: str                      #
    "acquisition","validation","segmentation","ai_handoff","compile",...
    status: Literal["ok","warn","error","skipped","not_run"]
    started_at: str | None
    duration_s: float | None
    metrics: list[Metric] = []
    details: dict[str, Any] = {}    # free-form raw blob (provenance)
    messages: list[str] = []

class Flag(BaseModel):            # a footgun turned into an actionable signal
    code: str                      # stable id, e.g. "silent_provider_noop"
    severity: Literal["error","warn","info"]
    message: str
    stage: str | None
    faq_ref: str | None            # e.g. "README#Q7" ? REQUIRED for debuggability eval
    evidence: dict[str, Any] = {}

class VideoMeta(BaseModel):       # customer-facing facts
    duration_s: float | None
    fps: float | None
    fps_source: Literal["probed","fallback","declared","unknown"] = "unknown"
    resolution: str | None
    width: int | None
    height: int | None
    container: str | None
    size_mb: float | None

class HighlightSummary(BaseModel): # meta-level, customer-safe
    segment_count: int | None
    total_highlight_s: float | None
    coverage_pct: float | None
    notes: list[str] = []

class ReportEnvelope(BaseModel):
    identity: Identity
    video_meta: VideoMeta
    stages: list[Stage] = []
    flags: list[Flag] = []
    highlights: HighlightSummary | None = None
    verdict: Literal["pass","pass_with_warnings","fail"] = "pass"
    extras: dict[str, Any] = {}    # escape hatch; never required by renderers
...

```

****Audience views**** are derived, not stored twice: the customer renderer takes the SAME envelope and emits only ``video_meta``, ``highlights``, and ``flags`` with ``severity!=info & audience-safe``, plus ``verdict``. Technical renderer emits everything. This guarantees the two reports can never disagree on facts (single source of truth) ? directly serves goal (a) catch inconsistencies and (b) cross-run/cross-tool diffing.

2.3 Declarative config language (``spec.py``, YAML)

A spec describes *sections* (how to lay out a report), *which fields are customer-visible*, and *rules*. It is data, loaded into pydantic models. Example (``specs/indexer.yaml``):

```

```yaml
schema_version: "1.0"
tool: badminton-highlight-indexer
sections:

```

```

- id: video_meta
 title: "Video"
 audience: both
 fields: [duration_s, fps, fps_source, resolution, container, size_mb]
- id: validation
 title: "Validation"
 audience: technical
 from_stage: validation
- id: segmentation
 title: "Segmentation"
 audience: technical
 from_stage: segmentation
- id: highlights
 title: "Highlights"
 audience: customer
rules:
- code: silent_provider_noop
 severity: error
 when:
 # ALL clauses must hold (AND)
 - { path: "stages.ai_handoff.details.api_key_present", op: "eq", value: false }
 - { path: "stages.segmentation.metrics.segment_count", op: "eq", value: 0 }
 message: "Segmenter produced 0 segments and no API key was present ? likely a silent
provider no-op, not 'no rallies'."
 faq_ref: "README#Q7"
- code: fps_fallback_used
 severity: warn
 when:
 - { path: "video_meta.fps_source", op: "eq", value: "fallback" }
 message: "fps was not probed; fell back to 30.0 ? velocity thresholds may be mis-scaled."
 faq_ref: "README#Q5"
- code: synthetic_motion_metadata
 severity: warn
 when:
 - { path: "stages.segmentation.details.segmenter", op: "eq", value: "motion" }
 message: "motion segmenter FABRICATES events/metadata (random) ? not ground truth."
 faq_ref: "docs/CODE_MAP.md#motion"
- code: segmenter_divergence
 severity: info
 when:
 - { path: "stages.segmentation.details.segmenter_requested", op: "ne_path", value:
"stages.segmentation.details.segmenter_actual" }
 message: "Configured default segmenter differs from the one that ran."
...

```

The spec is intentionally NOT Turing-complete. It can only: pick sections, tag audience, and evaluate restricted rule clauses.

## 2.4 Rule engine (rules.py) ? restricted & safe

```

- A rule = `{code, severity, when:[clause], message, faq_ref}`.
- A clause = `{path, op, value}`. `path` is a dotted accessor into the serialized envelope
(helper does safe nested `.get`, missing path ? clause is False unless `op` is `is_missing`).
- Supported ops only: `eq, ne, gt, ge, lt, le, in, contains, is_missing, is_present,
ne_path/eq_path` (compare two paths). NO arbitrary expressions, NO `eval`. Each op is a tiny
pure function ? trivially unit-testable.
- `when` = AND of clauses (keep simple; if OR ever needed, add a sibling `when_any`). Engine
returns `list[Flag]`. Verdict derivation: any `error` ? `fail`; any `warn` (no error) ?
`pass_with_warnings`; else `pass`.

```

This is the heart of "turn footguns into flags." Every flag carries `faq\_ref` so the customer/technical report cross-references README/FAQ ? the debuggability requirement.

## 2.5 RunRecorder (recorder.py) ? two capture modes

```

Mode A ? in-flight incremental (preferred):

```

```

...
rec = RunRecorder(identity=..., spec=load_spec("indexer"))
rec.set_video_meta(duration_s=..., fps=..., fps_source="probed", ...)
with rec.stage("validation") as st:
 st.add_metric("passed_count", n); st.detail("results", [...])
with rec.stage("segmentation") as st:
 st.detail("segmenter_requested", req); st.detail("segmenter_actual", act)
 st.add_metric("segment_count", len(segs))
env = rec.finalize() # runs rule engine -> flags + verdict
rec.write(base_dir=..., override=...) # writes JSON + technical MD + customer MD
...

- `stage()` is a context manager: auto-times, auto-sets status (`error` if exception, re-raises
only if caller wants ? default: record + swallow so reporting never breaks the run; see §7).
- `finalize()` applies the spec's rules to the assembled envelope.

```

**\*\*Mode B ? post-hoc harvest (fallback / backfill):\*\***

```

...
env = harvest_indexer_run(db, video_id, video_path, config, segments, val_results)
...

```

Pure functions in `recorder.py` (or a `harvest/` submodule) that reconstruct an envelope from already-persisted artifacts: DB candidates (`query\_candidate\_segments`), `play\_segments`, WASB `manifest.json`, compiler skip tuples. This guarantees a report can ALWAYS be produced even if a stage forgot to record in-flight, and enables backfilling old runs for cross-run diffing. The indexer should use Mode A on the live path and Mode B as the safety net inside the same write call (fill gaps from DB if a stage is missing).

## 2.6 Renderers

- `render/json.py`: `env.model\_dump\_json(indent=2)` ? the source of truth. Deterministic key order (pydantic preserves field order); sort lists by stable keys so cross-run diffs are clean.
- `render/markdown\_technical.py`: Jinja2 template, every stage + metrics + details + all flags with FAQ refs + verdict. Hand-written wording.
- `render/markdown\_customer.py`: Jinja2, only `video\_meta`, `highlights`, customer-safe flags, verdict. Optional HTML twin via a second template (or `markdown` lib ? but to stay dep-light, ship MD and let callers convert; HTML can be a Phase 3 add with a tiny template, no new dep).

## 2.7 io.py ? path convention + safe write

- Default: next to the video. For a video at `/x/match.mp4`:
  - `/x/match.report.json`, `/x/match.report.md` (technical), `/x/match.summary.md` (customer).
- Override: `report.output\_dir` config key ? same basenames under that dir.
- **\*\*Remote-only (collector, no local file)\*\***: fall back to `report.output\_dir` or a per-tool default (`reports/<video\_id>/...`). Never crash on missing local path.
- **\*\*Safe write\*\***: write to temp + atomic rename; wrap ALL writes in try/except that logs and returns a result object ? NEVER raises into the caller (§7).
- Naming uses `report.`/`.summary.` infixes to avoid colliding with the video and to be greppable.

## 2.8 Install across two private repos (git+ssh)

- Each repo adds to its deps:
  - Indexer `requirements.txt`: `obsreport @ git+ssh://git@github.com/<org>/sports-obsreport.git@v0.1.0`
  - Collector `pyproject.toml`: `"obsreport @ git+ssh://git@github.com/<org>/sports-obsreport.git@v0.1.0"`
- **\*\*Pin to a tag\*\*** (not a branch) so a library change can't silently break either repo ? bump deliberately. This is the primary house-of-cards guard between repos.
- Offline tests: lib is installed in the venv like any dep; both repos already run pytest offline with mocked externals ? `obsreport` has no network/IO at import, so it's mock-free.

---

## 3. Indexer integration (Repo A) ? exact hooks

### 3.1 Surface fprobe\_meta from run\_all (backward-compatible)

`backend/validators/base.py` `run\_all` discards `context`. **\*\*Recommended approach (non-breaking):\*\*** add an optional sibling method, do NOT change `run\_all`'s signature/return (tests +

```
both callers depend on `List[ValidationResult]`.
...

```

```
def run_all_with_context(self, video_path, config) -> tuple[list[ValidationResult], dict]:
 # same loop, returns (results, context)
 ...

```

Then `run\_all` becomes `return self.run\_all\_with\_context(...)[0]` (keeps existing callers and tests green). The two callers in `main.py` switch to `run\_all\_with\_context` to capture `context['ffprobe\_meta']` for the report. This avoids re-probing and avoids touching the public contract.

- Fallback if `ffprobe\_meta` absent (validator threw): recorder marks `fps\_source="unknown"` and the harvest re-probes via `backend/utils/video.get\_video\_duration` + cv2 (the same `\_probe\_fps\_width` fallback the segmenters use) ? and flags `fps\_fallback\_used` if fallback values were hit.

### 3.2 Where to instantiate RunRecorder (BOTH paths)

- **CLI** `backend/main.py:312-352`: create `rec` right after `video\_id = get\_file\_md5(...)` . Wrap validation, add\_video, segmentation each in `rec.stage(...)` . After `segments = segmenter.process\_video(...)` , call harvest to fill candidates/AI-timing/WASB/compiler, then `rec.write(...)` in a `finally` so a failed run STILL emits a report (records partial stages + the error flag).
- **API** `process\_video @ main.py:88-134`: same, but note there are TWO return points (the validation-failure early return at :104-109, and the success return at :128-134). Instantiate `rec` after `video\_id`; on the failure branch, finalize+write before returning; on success, finalize+write before returning. Best done by wrapping the body so both exits go through one write. The report path(s) should be added to the JSON response (`"reports": {...}`) so API callers can find them.
- `report.\*` config read from `global\_config` (CLI merges `output\_dir` into config at :302-305 ? mirror with a `report` block; default behavior = next-to-video).

### 3.3 Harvest sources (Mode B helpers)

- **Probe meta**: `context['ffprobe\_meta']` (preferred) else re-probe.
- **Validation**: the `List[ValidationResult]` already in hand ? one Stage with per-validator metrics/details; flags for any `passed=False`.
- **Segmentation**: `segments` return value ? `segment\_count`, total duration. `segmenter\_requested` = config default key; `segmenter\_actual` = the `seg\_name` actually resolved (encode the three-defaults divergence: config.json `gemini` / setup.py `yolo\_hybrid` / main.py `motion`).
- **Candidate windows**: `db.query\_candidate\_segments(video\_id, detector=...)` ? counts, velocity stats, detection\_params.
- **AI timing**: NOT currently returned. Two options:
  - (Phase-later, clean) thread the `metrics` dict out of `analyze\_video` through `process\_video` into the recorder. Requires touching `process\_video` return or adding an optional `recorder`/`on\_metrics` callback param to the segmenter ? invasive across 3 twin segmenters.
  - (Phase-first, cheap) harvest what's persisted (candidate counts, WASB manifest timings) and record AI timing as `not\_captured` with a note. Recommend starting cheap; add the callback in a later PR only for the live path.
- **WASB manifest**: read `output/wasb/manifest.json` if present ? `{frames\_total, windows\_total, windows\_done}` into the segmentation/ai stage details. Guard: relative cwd path; resolve against the run's output dir.
- **Compiler skips**: `VideoCompiler.compile\_highlights` returns skipped tuples `(idx,start,end,reason)` ? compile is a SEPARATE endpoint/step, so its report is a separate concern; for the per-process report, record compile as `not\_run`. (If/when compile runs, emit/append a compile stage ? see idempotency §7.)

### 3.4 Footgun flags to encode (indexer spec)

```
| code | trigger | severity | faq_ref |
|---|---|---|---|
| `silent_provider_noop` | 0 segments AND api_key_present==false (hybrids/gemini) | error | README#Q7 |
| `ai_empty_vs_no_rallies` | provider returned [] on a non-empty candidate set | warn | README#Q7 |
| `fps_fallback_used` | fps_source==fallback (30.0/1280.0) | warn | README#Q5 |
| `synthetic_motion_metadata` | segmenter==motion | warn | CODE_MAP#motion |

```

```
| `segmenter_divergence` | requested != actual segmenter | info | CODE_MAP $three defaults |
| `reingest_cascade` | add_video INSERT OR REPLACE replaced an existing video_id (prior segments
cascade-deleted) | warn | CODE_MAP#add_video |
| `validation_failed` | any ValidationResult.passed==false | error | README#Q2/Q5 |
| `wsl_healthcheck_failed` | WASB/tracknet healthcheck not ok ? [] | error | README#Q7 |

```

## 4. Collector integration (Repo B) ? wrap, don't duplicate

The collector already HAS the report primitives in `src/parsers/cvat/rally\_cvat.py`. Strategy: **\*adapter, not rewrite.\*** Add a thin `src/reporting/envelope\_adapter.py` that maps existing outputs into the shared envelope; keep `build\_summary/detect\_issues/render\_validation\_md` as the authoritative content producers.

```
- **Issue?Flag mapping**: `Issue{severity in (blocker,high,medium,low), code, message,
rally_number}` ? `Flag`:
 - `blocker`?`error`, `high/medium`?`warn`, `low`?`info`.
 - `code` carried verbatim; `faq_ref` mapped per-code (collector has its own docs/FAQ; add a
small code?faq table).
 - `evidence={"rally_number": ...}`.
- **Per-stage model for the collector** (the "stages a video went through"):
 - `acquisition` (fetch): from `DownloadResult` (downloader.py:143) ? url, attempts, size_mb,
probed width/height/fps, error. For import-local/convert-cvat there is no download ? stage
`not_run` or `skipped`.
 - `parse`: source format (CSV/JSON/CVAT), n records (from `ConversionResult.records` or
`_parse_local_annotations`).
 - `time_recompute`: fps used + frame range + step (CVAT path:
`ConversionResult.fps/step/meta`). Flag if fps came from a guess (the very bug rally_cvat exists
to undo).
 - `validation`: existing `detect_issues` / sport validators ? flags.
 - `registration`: DB insert result (video_id, status, license, is_commercial).
 - `export`: per-video CSV + manifest (from `export_eval_set`).
- **Wire-in points** (each writes the envelope after its existing work, never replacing it):
 - `import_local` (`curate.py:52-142`): after insert (:124), build envelope from annotations +
validation + VideoMetadata; write next to `args.video_path` (or override).
 - `convert_cvat` (`curate.py:526-560`): already produces `ConversionResult` + issue report;
ADD envelope write alongside the existing CSV/issues output.
 - `validate_delivery` (`curate.py:562-617`): already writes `_report.json/.md`. Reuse
`build_summary` as the content, wrap into envelope, ALSO write the customer summary. Keep the
existing files for back-compat (or make the new envelope the canonical `_report.json` in a
deliberate, tested migration).
 - `fetch` (`curate.py:330+`): per video, from `DownloadResult`, write an acquisition-focused
report.
 - `export` (`curate.py:428-524`): the manifest is corpus-level; per-video reports are better
emitted at ingest. Optionally add a customer "delivery summary" per exported video here.
- **Customer "delivery summary"**: `video_meta` (from VideoMetadata: fps, resolution,
match_name, license/commercial) + `HighlightSummary` (n rallies, total rally seconds, coverage)
+ only `error`/`warn` flags phrased for a non-engineer. No internal codes in the customer view
(strip `code`, keep `message`).
- **New dep**: collector gains `obsreport` (which brings Jinja2). PyYAML already present.
```

---

## 5. README/FAQ-reader debuggability ? concrete eval method

Make "a person with only README+FAQ can self-debug" a *\*measurable, iterated\** gate.

- \*\*Fixtures of deliberately-broken runs\*\*** (offline, mocked): e.g. (a) no API key ? `silent\_provider\_noop`; (b) skipped probe ? `fps\_fallback\_used`; (c) motion segmenter ? `synthetic\_motion\_metadata`; (d) re-ingest ? `reingest\_cascade`; (e) collector overlap/zero-duration ? blocker flags. Each fixture produces a real report via the harness.
- \*\*"Fresh reader" subagent eval\*\***: spawn a subagent given ONLY `README.md` + the FAQ + the generated technical report (NO source). Task: "diagnose the root cause and name the fix." Pass =

it identifies the correct cause and cites the FAQ section the flag points to.

3. **\*\*Iterate wording\*\***: if the subagent fails, adjust the flag `message` and `faq\_ref` (and, if needed, the FAQ text) until diagnosis succeeds. Each flag MUST cite a FAQ/CODE\_MAP section (`faq\_ref` is a required field for `error`/`warn` flags ? enforce in a test).
4. **\*\*Lock as regression\*\***: store the broken-run fixtures + expected `{flag codes, faq\_refs}` as golden files; a pytest asserts the report contains exactly those flags. This prevents wording/flag drift and makes the debuggability property durable.
5. **\*\*Cross-tool/cross-run diff test\*\***: run the same logical video through both tools (or two indexer runs) and assert the JSON envelopes diff only where expected ? exercises goals (b)/(c).

---

## 6. Phasing into small PRs (3 repos), value-early & de-risked

**\*\*Riskiest parts\*\***: (i) the cross-repo dependency (a lib bug breaks both tools) ? contained by tag-pinning + the lib being pure/no-IO-at-import; (ii) touching `run\_all` ? contained by the additive `run\_all\_with\_context`; (iii) AI-timing threading through 3 twin segmenters ? deferred and isolated.

**\*\*Repo C (library) ? land first, independently testable:\*\***

- **\*\*PR C1\*\***: repo scaffold + `envelope.py` + `version.py` + JSON renderer + tests (envelope round-trips, schema\_version present, deterministic JSON). No consumers yet. *\*Value: the contract exists.\**
- **\*\*PR C2\*\***: `spec.py` + `rules.py` (restricted ops) + unit tests per op + verdict derivation. Ship `specs/indexer.yaml` + `specs/collector.yaml` (data only).
- **\*\*PR C3\*\***: `render/markdown\_technical.py` + `markdown\_customer.py` + Jinja2 templates + snapshot tests. `io.py` path resolution + safe-write + tests (next-to-video, override, remote-no-file).
- **\*\*PR C4\*\***: `recorder.py` RunRecorder (Mode A) + harvest scaffolding (Mode B interfaces) + tests. Tag **\*\*v0.1.0\*\***.

**\*\*Repo A (indexer) ? consume incrementally:\*\***

- **\*\*PR A1\*\***: add `obsreport @ v0.1.0` dep; add `run\_all\_with\_context` (additive, keep `run\_all`); tests prove `run\_all` unchanged + context surfaced. *\*No behavior change to reports yet.\**
- **\*\*PR A2\*\***: Mode-B harvest helper `harvest\_indexer\_run(...)` (pure; reads DB candidates/play\_segments/probe meta) + tests with a mocked DB. Still not wired into main.
- **\*\*PR A3\*\***: wire RunRecorder into the **\*\*CLI path\*\*** only (`main.py:312-352`), write next-to-video, `finally`-guarded. Add indexer spec flags (Q-cross-refs). Tests: CLI run emits 3 files, broken-run fixtures produce expected flags.
- **\*\*PR A4\*\***: wire into the **\*\*API path\*\*** (both return branches) + add report paths to the JSON response. Tests for both branches.
- **\*\*PR A5\*\*** (optional, later): thread AI-timing `metrics` out of `analyze\_video` via a recorder callback (one segmenter first, then mirror to the twins). Isolated, opt-in.

**\*\*Repo B (collector) ? consume incrementally:\*\***

- **\*\*PR B1\*\***: add `obsreport` dep; `src/reporting/envelope\_adapter.py` (Issue?Flag, stage builders) + tests using existing `ConversionResult` fixtures. Not wired.
- **\*\*PR B2\*\***: wire `validate\_delivery` (lowest risk ? it already writes a report) to ALSO emit the envelope JSON + customer summary, keeping existing files. Tests.
- **\*\*PR B3\*\***: wire `import\_local` + `convert\_cvat`. Tests.
- **\*\*PR B4\*\***: wire `fetch` acquisition report + optional `export` delivery summary. Handle remote-no-local-file path. Tests.

**\*\*Debuggability eval (cross-cutting):\*\***

- **\*\*PR D1\*\*** (after A3/B2): broken-run fixtures + "fresh reader" eval harness + golden flag/faq assertions. Run in CI for both repos.

Ordering rationale: library tagged before any consumer; each consumer PR is independently shippable and reversible; the highest-value/lowest-risk consumer wiring (`validate\_delivery`, indexer CLI) lands before the trickier API/twin-segmenter work.

---



## 7. Risks & edge cases (and mitigations)

- **Report write must NEVER break the main run.** All recorder writes wrapped in try/except returning a result object; in main.py, write in a `finally`; a write failure logs a warning and is itself recorded as an `extras.report\_write\_error` on the next successful surface ? but never raises. Add a test that injects an IOError and asserts the pipeline result is unaffected.
- **Remote videos / no local file (collector).** `io.py` resolves to `report.output\_dir` or `reports/<video\_id>/` when `local\_path` is None/missing; `video\_meta` carries `video\_url`; never assume a local path. Covered by a dedicated test.
- **Schema versioning.** `SCHEMA\_VERSION` in `identity`; renderers assert major-version compatibility. Bump = deliberate, with a changelog. Consumers pin a lib tag so a schema bump can't surprise them. Old JSON reports remain readable (additive fields only within a major).
- **Idempotency / overwrite vs append.** Default = overwrite per `run\_id` (latest run wins next to the video) ? keeps the convention clean and matches `add\_video` INSERT-OR-REPLACE semantics. For audit/cross-run diff, support an optional `report.history\_dir` that archives `- **Large reports.** `details` blobs (e.g. all candidate windows, full trajectory) can be big. Mitigation: cap list lengths in the technical renderer (e.g. first N + "? (M more)"), keep full data only in JSON, and provide a `report.max\_detail\_items` knob. Customer report never includes raw lists.
- **Cross-repo coupling (house of cards).** Tag-pin the lib in both repos; lib is pure (no IO at import, no network); a CI job in the lib repo runs both consumers' relevant tests against `HEAD` before tagging (contract test). A breaking lib change is caught before either repo bumps.
- **Twin-segmenter drift** (wasb\_hybrid/tracknet\_hybrid are copy-paste): if AI-timing threading is added (A5), apply to both; harvest-from-DB (A2) avoids touching them at all initially ? prefer that.
- **video\_id semantics differ** (indexer=file MD5, collector=human id): `identity.video\_id\_kind` records which, so cross-tool diffs don't silently mis-join. Documented in the envelope.
- **cwd-relative paths** (WASB `output/wasb`, manifest): harvest resolves against the run's configured output dir, not raw cwd, to avoid empty reports when run from a different directory.

---

## Critical Files for Implementation

- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py (CLI :312-352 + API :88-134 recorder hooks)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\validators\base.py (add run\_all\_with\_context to surface fprobe\_meta)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\indexer\wasb\_hybrid.py (candidate/AI-timing/WASB-manifest harvest reference; twin = tracknet\_hybrid.py)
- C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally\_cvat.py (reuse build\_summary/detect\_issues/Issue model; wrap into envelope)
- C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py (import\_local/convert\_cvat/validate\_delivery/fetch/export wire-in points)

35. user (2026-06-07T06:55:48.586Z)

<tool\_use\_error>Error: No such tool available: Write. Write exists but is not enabled in this context. Use one of the available tools instead.</tool\_use\_error>

36. assistant (2026-06-07T06:57:07.779Z)

Write isn't available in this read-only planning context. I'll deliver the plan directly as my response, which is what the parent agent reads.

I explored both repos and verified every load-bearing fact in your brief against the actual code. The plan below is hardened, file-level concrete, and phased.

---

## Rich Per-Video Observability Report ? Implementation Plan

## 0. Facts I verified in code (not assumed)

- **Indexer already has the deps**: `requirements.txt` has `pydantic>=2.7.1` and `jinja2>=3.1.4`. Collector `pyproject.toml` has `pydantic>=2.0.0`, `pyyaml>=6.0`, `tabulate` ? no Jinja2 (the shared lib brings it transitively, BSD, fine).
- **run\_all discards context**: `backend/validators/base.py:42-56` builds a local `context={}`, `FormatValidator` fills `context['ffprobe\_meta']`, and only `List[ValidationResult]` is returned. Confirmed: to get probe metadata you must surface context or re-probe.
- **AI timing is dropped at the segmenter**: `backend/indexer/wasb\_hybrid.py:167` is literally `segments, \_ = provider.analyze\_video(...)` ? the `{upload\_time, process\_time, gen\_time}` dict is thrown away. Same in the twins.
- **Candidates ARE persisted**: `wasb\_hybrid.py:144` calls `db.add\_candidate\_segment(...)`; harvestable via `db.query\_candidate\_segments(detector=)`. WASB manifest at `output/wasb/manifest.json` (cwd-relative).
- **Both entrypoints confirmed**: CLI `backend/main.py:312-352`, API `process\_video @ main.py:88-134` (note the API has TWO return points: validation-fail at :104-109 and success at :128-134).
- **Collector report module to reuse**: `src/parsers/cvat/rally\_cvat.py` ? `build\_summary()` (JSON dict), `render\_validation\_md()`, `write\_review\_csv()`, `detect\_issues()` with `Issue{severity in blocker/high/medium/low, code, message, rally\_number}`, `ConversionResult{records, issues, fps, step, meta}`. `validate\_delivery` (`curate.py:562-617`) already writes `\_report.json/.md` + `review.csv`.
- **Collector Video model** `src/core/schemas/base.py:29-40`: `video\_id, video\_url, local\_path, license\_type, is\_commercial, status, match\_name, fps, resolution`. `probe\_resolution\_fps` + `DownloadResult{success, size\_mb, width, height, fps, resolution\_label, error}` in `src/core/downloader.py`.
- **License posture (ADR-005, docs/DECISIONS.md:232)**: hard permissive-only; pydantic/Jinja2/PyYAML/jsonschema all clean.
- **FAQ Q1-Q9** (`README.md:166+`) maps directly to footgun flags: Q5?PTS/keyframe & fps, Q7?AI provider/API key, Q2?relevance, Q8?yolo economics. These are the `faq\_ref` citation targets.

---

## 1. Library-vs-framework decision ? VALIDATED

Build the small in-house library. It is the right call, and not only for licensing:

1. **Shape mismatch is decisive**. GE/Evidently/ydata/Frictionless model \*tabular datasets / data packages\*. Your domain object is a **per-video run** = an ordered list of heterogeneous pipeline stages, each with a free-form metrics blob and a verdict, plus a **declarative footgun?flag rule layer**. None of those tools expresses that naturally; forcing it in becomes its own house of cards.
2. **License is an independent hard gate** (ADR-005) that already kills Soda (ELv2).
3. **House-of-cards containment**: ~6-8 tiny modules, a frozen pydantic envelope, pure renderers, no IO at import. Far less fragile than wrapping a heavy framework whose API churn you don't control.

**Refinements to your stated stack**:

- **Drop `jsonschema` from the runtime core** (defer to Phase 2+). pydantic v2 is your single source of truth and emits JSON Schema via `model\_json\_schema()`. Only add `jsonschema` if/when you validate \*externally authored\* spec files against a published schema.
- **Do NOT take `json-schema-for-humans`/`yaml-to-markdown` as runtime deps**. Hand-write the Jinja2 templates ? the FAQ-debuggability eval \*requires\* you to control exact wording. Treat those MIT tools as optional dev helpers for generating the schema doc only.
- **Keep the "config language" sub-Turing**. No `eval`, no DSL parser. A restricted list of typed comparison clauses (§2.4) covers every footgun you listed.

---

## 2. The shared library

### 2.1 Repo / package name

Repo **sports-obsreport**, package **obsreport**, private GitHub, MIT. (Use generic `obsreport` package name so non-sports reuse later is clean.)

```

...
src/obsreport/
 envelope.py spec.py rules.py recorder.py io.py version.py
 render/{json.py, markdown_technical.py, markdown_customer.py, templates/*.j2}
specs/{indexer.yaml, collector.yaml}
tests/
...

```

## 2.2 Typed envelope (`envelope.py`, pydantic v2) ? single source of truth

Both Markdown renderers consume the SAME envelope, so the two report types can never disagree on facts (serves goal a: catch inconsistencies).

```

- `Identity{schema_version, tool, tool_version, video_id, video_id_kind ("file_md5"|"human_id"),
video_path, video_url, generated_at, run_id}`
- `Metric{key, value, unit, audience: technical|customer|both}`
- `Stage{name, status: ok|warn|error|skipped|not_run, started_at, duration_s, metrics[],
details: dict, messages[]}`
- `Flag{code, severity: error|warn|info, message, stage, faq_ref, evidence}` ? `faq_ref`
REQUIRED for error/warn (enforced by test).
- `VideoMeta{duration_s, fps, fps_source: probed|fallback|declared|unknown, resolution, width,
height, container, size_mb}`
- `HighlightSummary{segment_count, total_highlight_s, coverage_pct, notes[]}` ? customer-safe,
meta-level only.
- `ReportEnvelope{identity, video_meta, stages[], flags[], highlights, verdict:
pass|pass_with_warnings|fail, extras}`

```

**\*\*Audience views are derived, not stored twice.\*\*** Customer renderer emits only `video\_meta + highlights + customer-safe flags + verdict`; technical renderer emits everything. One source of truth = the two reports physically cannot diverge.

## 2.3 Declarative config language (`spec.py`, YAML, loaded into pydantic)

A spec = `sections[]` (layout + audience + which fields/stage) and `rules[]`. Pure data. Example footgun rule:

```

```yaml
rules:
  - code: silent_provider_noop
    severity: error
    when:
      - {path: "stages.ai_handoff.details.api_key_present", op: eq, value: false}
      - {path: "stages.segmentation.metrics.segment_count", op: eq, value: 0}
    message: "0 segments AND no API key ? likely a silent provider no-op, not 'no rallies'."
    faq_ref: "README#Q7"
...

```

2.4 Rule engine (`rules.py`) ? restricted & safe

- Clause `{path, op, value}`; `path` = dotted accessor into serialized envelope (missing path ? False unless `op=is\_missing`).
- Ops only: `eq, ne, gt, ge, lt, le, in, contains, is\_missing, is\_present, eq\_path, ne\_path`. Each is a tiny pure function. NO `eval`, NO expression grammar.
- `when` = AND of clauses (add `when\_any` later only if OR ever needed).
- Verdict: any `error`? `fail`; any `warn` (no error)? `pass\_with\_warnings`; else `pass`.

2.5 RunRecorder (`recorder.py`) ? two modes

- ****Mode A (in-flight)****: `rec.stage("segmentation")` context manager auto-times, auto-sets status, records exceptions WITHOUT re-raising by default (so reporting never breaks the run). `rec.finalize()` runs the rule engine; `rec.write(...)` emits all 3 files.
- ****Mode B (post-hoc harvest)****: pure functions reconstruct the envelope from persisted artifacts (DB candidates/play_segments, WASB manifest, compiler skips). Guarantees a report is ALWAYS produceable and enables backfilling old runs for cross-run diff (goal b/c). The indexer uses A live and B as the gap-filler inside the same write.

2.6 Renderers

- `json.py`: `model\_dump\_json(indent=2)`, deterministic ordering + sorted lists ? clean cross-run diffs.

- `markdown\_technical.py` / `markdown\_customer.py`: hand-written Jinja2. HTML customer twin deferrable to a later PR with a second template (no new dep).

2.7 `io.py` ? convention + safe write

- Next-to-video default: `/x/match.mp4` ? `match.report.json`, `match.report.md` (technical), `match.summary.md` (customer).
- Override via `report.output\_dir` config key.
- Remote/no-local-file (collector): fall back to `report.output\_dir` or `reports/<video\_id>/`. Never crash on missing local path.
- Atomic temp+rename; ALL writes wrapped to return a result object, never raise (see S7).

2.8 Install across two private repos (git+ssh)

- Indexer `requirements.txt`: `obsreport @ git+ssh://git@github.com/<org>/sports-obsreport.git@v0.1.0`
- Collector `pyproject.toml` dep: same `@v0.1.0`.
- ****Pin to a tag, never a branch**** ? this is the primary cross-repo house-of-cards guard. Bump deliberately. Lib has no network/IO at import, so it adds nothing to mock in either repo's offline pytest.

3. Indexer integration ? exact hooks

****3.1 Surface ffprobe_meta (backward-compatible).**** Add `run\_all\_with\_context(...)` -> (list[ValidationResult], dict) to `backend/validators/base.py`; make `run\_all` call it and return `[0]`. Existing callers and tests stay green. The two `main.py` callers switch to `run\_all\_with\_context` to grab `context['ffprobe\_meta']` ? no re-probe, no contract break. Fallback: if missing, harvest re-probes (same cv2 fallback the segmenters use) and raises `fps\_fallback\_used`.

****3.2 RunRecorder placement.****

- CLI (`main.py:312-352`): create `rec` after `video\_id = get\_file\_md5(...)`; wrap validation/add_video/segmentation in `rec.stage(...)`; `rec.write(...)` in a `finally` so even a failed/exception run emits a partial report with the error flag.
- API (`main.py:88-134`): instantiate after `video\_id`; route BOTH exits (fail :104-109, success :128-134) through one finalize+write; add report paths to the JSON response (`"reports": {...}`).

****3.3 Harvest sources (Mode B):**** probe meta from context; validation from the `List[ValidationResult]` in hand; segmentation from `segments` (+ `segmenter\_requested` vs `segmenter\_actual`); candidates from `db.query\_candidate\_segments`; WASB `{frames\_total, windows\_total, windows\_done}` from `output/wasb/manifest.json` (resolve against configured output dir, not raw cwd); compiler skips deferred (compile is a separate endpoint ? record `not\_run`). AI timing: start by recording `not\_captured` (cheap); add a recorder callback through `analyze\_video` ? `process\_video` in a LATER isolated PR (touches the twins, so defer).

****3.4 Indexer flags:**** `silent\_provider\_noop` (error, Q7), `ai\_empty\_vs\_no\_rallies` (warn, Q7), `fps\_fallback\_used` (warn, Q5), `synthetic\_motion\_metadata` (warn, CODE_MAP#motion), `segmenter\_divergence` (info ? the three-defaults footgun), `reingest\_cascade` (warn ? INSERT OR REPLACE deleted prior segments), `validation\_failed` (error, Q2/Q5), `wsl\_healthcheck\_failed` (error, Q7).

4. Collector integration ? wrap, don't duplicate

Add a thin `src/reporting/envelope\_adapter.py`; keep `build\_summary/detect\_issues/render\_validation\_md` as the authoritative content producers.

- ****Issue?Flag****: `blocker?error`, `high/medium?warn`, `low?info`; carry `code` verbatim; map `code?faq\_ref` via a small table; `evidence={"rally\_number":...}`.
- ****Stages****: `acquisition` (from `DownloadResult`), `parse` (format + n records), `time\_recompute` (CVAT `fps/step/meta` ? flag guessed fps, the exact bug rally_cvat exists to undo), `validation` (existing `detect\_issues`/sport validators), `registration` (DB insert),

```
`export`.
- Wire-in points (add envelope write AFTER existing work, never replacing): `import_local`
(`curate.py:52-142`, after insert :124), `convert_cvat` (:526-560), `validate_delivery`
(:562-617` ? lowest risk, already writes a report; reuse `build_summary` content, add customer
summary, keep existing files for back-compat), `fetch` (:330+, acquisition report), `export`
(:428-524`, optional per-video delivery summary).
- Customer "delivery summary": `VideoMeta` (fps/resolution/match_name/license) +
`HighlightSummary` (n rallies, total rally seconds, coverage) + only error/warn flags with
`code` stripped, message kept.
```

5. README/FAQ-reader debuggability ? concrete eval loop

- Broken-run fixtures** (offline, mocked): no-API-key, skipped-probe, motion-segmenter, re-ingest, collector overlap/zero-duration. Each emits a real report.
- "Fresh reader" subagent**: given ONLY README+FAQ+the technical report (no source), must name the root cause and the fix. Pass = correct cause + cites the FAQ section the flag's `faq\_ref` points to.
- Iterate** flag `message`/`faq\_ref` (and FAQ text if needed) until diagnosis succeeds. Enforce `faq\_ref` required for error/warn via a test.
- Lock as golden regression**: fixture ? expected `{flag codes, faq\_refs}`; pytest asserts exact set. Prevents wording/flag drift.
- Cross-tool/cross-run diff test**: same logical video through both tools / two runs ? JSON envelopes diff only where expected (goals b/c).

6. Phasing into small PRs

Riskiest: (i) cross-repo dep ? contained by tag-pin + pure lib; (ii) touching `run\_all` ? contained by additive `run\_all\_with\_context`; (iii) AI-timing through 3 twin segmenters ? deferred/isolated.

Library (land + tag first): C1 scaffold + envelope + JSON renderer + tests ? C2 spec + rules + verdict tests ? C3 MD renderers + io safe-write/path tests ? C4 RunRecorder + harvest scaffolding + tests ? **tag v0.1.0**.

Indexer: A1 add dep + `run\_all\_with\_context` (prove `run\_all` unchanged) ? A2 pure `harvest\_indexer\_run` (mocked DB) ? A3 wire CLI path (`finally`-guarded) + flags + broken-run tests ? A4 wire API (both branches) + report paths in response ? A5 (later) thread AI timing via callback, one segmenter then twins.

Collector: B1 add dep + `envelope\_adapter` (Issue?Flag, stage builders) + tests ? B2 wire `validate\_delivery` (lowest risk) ? B3 wire `import\_local` + `convert\_cvat` ? B4 wire `fetch` + remote-no-file handling + optional `export` summary.

Cross-cutting: D1 broken-run fixtures + fresh-reader eval + golden flag/faq assertions in CI for both repos (after A3/B2).

Order rationale: lib tagged before any consumer; every consumer PR is independently shippable and reversible; lowest-risk/highest-value wiring (`validate\_delivery`, indexer CLI) lands before the trickier API/twin work.

7. Risks & edge cases

- Report write must never break the run**: writes wrapped to return a result, called in `finally`; inject-IOError test asserts pipeline result unaffected.
- Remote videos / no local file**: `io.py` resolves to `report.output\_dir`/`reports/<video\_id>/`; `video\_meta.video\_url` carried; dedicated test.
- Schema versioning**: `SCHEMA\_VERSION` in identity; renderers assert major compatibility; additive-only within a major; consumers pin a tag so a bump can't surprise them.

- **Idempotency**: default overwrite per latest run next to the video (matches ``add_video`` INSERT-OR-REPLACE semantics); optional ``report.history_dir`` archives ``<video_id>/<run_id>.json`` for diffing; compile (separate step) writes a sibling ``*.compile.report.json`` to avoid read-modify-write races.
- **Large reports**: cap list lengths in the technical renderer (first N + "?M more"); full data only in JSON; ``report.max_detail_items`` knob; customer report never includes raw lists.
- **Cross-repo coupling**: tag-pin + pure lib + a contract-test CI job in the lib repo that runs both consumers' relevant tests against ``HEAD`` before tagging.
- **Twin-segmenter drift**: prefer harvest-from-DB (A2) so you don't touch wasb/tracknet copies at all initially; if A5 lands, apply to both.
- **``video_id`` semantics differ** (indexer file-MD5 vs collector human id): ``identity.video_id_kind`` records which so cross-tool diffs don't mis-join.
- **cwd-relative paths** (``output/wasb``): harvest resolves against the configured output dir, not raw cwd.

Critical Files for Implementation

- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py (CLI :312-352 + API :88-134 recorder hooks)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\validators\base.py (add ``run_all_with_context`` to surface `ffprobe_meta`)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\indexer\wasb_hybrid.py (candidate / AI-timing / WASB-manifest harvest reference; twin = `tracknet_hybrid.py`)
- C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py (reuse `build_summary/detect_issues/Issue`; wrap into envelope)
- C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py (import_local / convert_cvat / validate_delivery / fetch / export wire-in points)

Note: the plan file write was blocked (read-only planning context disables Write), so the full plan is delivered above for you to adapt into your final plan document.